JYOC - SProcesos - Ejercicios

Ejercicios de Servicios y Procesos

Separados por capítulos según temática

Iñaki Martín

INDICE

INDICE 2

Objetivos 3

Calificación de los ejercicios 3

Pautas para una buena programación de los ejercicios: 3

1. EJERCICIOS DE PROCESOS 4
2. EJERCICIOS DE HILOS 6
3. EJERCICIOS DE MONITORES SIMPLES (sin wait ni notify) 9
4. EJERCICIOS PRODUCTOR-CONSUMIDOR (o de monitores con wait-notify) 12
5. EJERCICIOS DE SEMÁFOROS 24
6. EJERCICIOS DE INTERRUPCIONES 31
7. EJERCICIOS DE SOCKETS 32
8. EJERCICIOS DE CRIPTOGRAFÍA 40
9. EJERCICIOS DE TEMATICA MULTIPLE 41

Objetivos

El interés de la resolución de estos ejercicios es estrictamente didáctica, por lo que se ha evitado en muchas ocasiones abreviar expresiones, se usan algoritmos sencillos y claros, y se proponen desarrollos más largos que eficientes, en aras a facilitar la comprensión y asimilación.

Sin embargo, se alienta al alumno a que mejore, afine y depure los ejercicios propios o estas soluciones, y que con ello se adquieran además prácticas de revisión y mejora.

Calificación de los ejercicios

Para agilizar el proceso formativo, se ha aplicado un factor de dificultad a los problemas, de modo que;

- ✤ El color verde indica ejercicios de nivel inicial
- ✤ El color azul indica ejercicios de nivel medio
- ✤ El color rojo indica ejercicios de nivel alto

Pautas para una buena programación de los ejercicios:

- ✦ Escribir el código con una correcta indentación y presentación. Se recomienda la utilización de estas opciones :
  - En Eclipse: Menu Source -> Format (atajo de de teclado: CTRL + SHIFT + F).
  - En NetBeans: Menu Source -> Format (atajo de de teclado: ALT + SHIFT + F).
  - En IntelliJ: Menu Code -> Reformat Code (atajo de de teclado: CTRL + ALT + L)
- ✦ Si hay atributos, no se deben declarar como públicos.
- ✦ Las operaciones de entrada de datos por teclado, acompañarlas de mensajes claros que soliciten la información al usuario.
- ✦ Usar métodos getters y setters sólo si son necesarios, los que no se necesiten, no se construyen.
- ✦ Deben usarse nombres significativos para los identificadores (nombres de clases y métodos, de variables y atributos). No llamar a una variable “x", si es mejor poner “cantidad”
- ✦ Preferible usar métodos auxiliares si estos implementen tareas comunes a varios métodos, y de esa forma, se puede evitar la duplicidad de código.
- ✦ Escribir código eficiente que no realice operaciones innecesarias.
- ✦ Esta desaconsejado el uso de return o break dentro de un bucle

1. EJERCICIOS DE PROCESOS

1.a.1. Crear un proceso que muestra por consola la ejecución de la instrucción "ls -l" (si usas Linux) o "dir" (si usas Windows).
Pista: en Windows, la llamada a "dir" no puede hacerse directamente, sino que hay que llamar realmente a la consola de Windows (cmd.exe) y pasarle la instrucción. En definitiva, el proceso que se lanza es "cmd.exe /C dir" . La parte /C indica que la instrucción "termine" cuando acabe su ejecución, si no, no se devuelve el control al proceso pues nunca "termina"

1.a.2. Crear un programa que nos muestre la siguiente información del sistema:
- Número de núcleos (cores) del ordenador.
- Memoria en Mb disponible y máxima en la Java Virtual Machine (jvm).
- Listado de todas las propiedades del sistema que ofrezca la clase Properties.

1.a.3. Crear un clase Java que escriba por consola 10 lineas, con distintos System.out.
Compilar la clase.
Crear otra clase que lance como un proceso la clase anterior, y muestre por pantalla las lineas que va escribiendo el primer proceso.

- MEJORA 1: cada proceso nos dice cuantas palabras encuentra que comiencen por A

1.a.4. Crear un programa que pide al usuario una dirección de una web, y usando la clase URL de Java, lea y muestre por consola el contenido HTML de dicha web

1.a.5. Escribe un programa llamado NumerosAleatorios que realice las siguientes tareas:
- Crea un proceso hijo que está encargado de generar números aleatorios. Este proceso hijo escribirá en su salida estándar un número aleatorio del 1 al 10 cada vez que se realice una petición de ejecución por parte del padre.
- El proceso padre lee líneas de la entrada estándar y por cada línea que lea, solicitará al hijo que le envíe un número aleatorio. Lo leerá e imprimirá por pantalla.
- Cuando el proceso padre reciba la palabra “fin”, finalizará la ejecución del hijo y procederá a finalizar su ejecución.

1.b. EJERCICIOS NIVEL MEDIO

1.b.1. Lanzado procesos Java desde otro proceso Java.
- Paso 1: crear una aplicación llamada *Primos* que escriba en un fichero de texto 10 números primos. Cada número se escribe en una línea distinta.
- Paso 2: modificar *Primos* para que el número de primos que se escriben y el nombre del fichero donde se escriben sean parámetros del proyecto (recibidos en el método main). Así, la llamada a este programa como un proceso externo sería algo así:

  ```
  java –jar Primos 12 FicheroPrimos.txt
  ```

  Este proyecto se puede probar desde Eclipse o Netbeans simulando el paso de parámetros
- Paso 3: compilar el proyecto *Primos* para crear el fichero compilado Primos.jar
- Paso 4: crear otra aplicación, llamada *Juntar*, que lance 10 instancias de la aplicación *Primos*, de modo que todas las ejecuciones creen un único fichero donde todos los procesos añadan números primos.

  Cada instancia generará un número creciente de palabras, cada uno 10 mas que el anterior: esto es, el primer proceso creará 10 primos, el segundo 20, el tercero 30, etc… Cada proceso seguirá

escribiendo sus números primos en una línea independiente de las otras y detrás de donde acabo el proceso anterior: Es decir, si lanzamos 10 instancias de *Primos,* al final debemos tener en el fichero 10 + 20 + 30 + … + 100 = 550 números primos.

1.b.2. Usando pipes para comunicar procesos

- Paso 1: Crear una aplicación que ordena 10 números que recibe a través de su entrada estándar; y muestra el resultado de la ordenación en su salida estándar. La aplicación se llamará 'OrdenarNumeros'.
- Paso 2: crear una aplicación, llamada 'EnviaNumeros', que genere 10 números aleatorios (entre 0 y 100), y que los escriba en su salida estándar.
- Paso 3: Ejecuta una instrucción desde una terminal (consola) del Sistema Operativo, que lance el programa "EnviaNumeros" y redireccione su salida hacia el programa "OrdenarNumeros"

PISTA: la ejecución simple del primer programa seria desde consola

```
java –jar EnviaNumeros.jar
```

Si queremos además reenviar la información de un proceso a otro, hay que usar pipes (tuberías) que comuniquen ambos procesos: la instrucción completa seria :

```
java –jar EnviaNumeros.jar | java –jar OrdenaNumeros.jar
```

1.b.3. Crear un programa que muestre los procesos que tiene el sistema y nos vaya preguntando por cada uno de ellos si lo queremos eliminar. Haz que funcione tanto en Windows como en Linux.

1.b.4. Escribe una clase llamada ProgramaPadre que se comunique con un proceso hijo mediante mensajes. El proceso padre enviará un mensaje por la entrada estándar del proceso hijo, y este le responderá por su salida estándar según el mensaje enviado. La tabla de mensajes y respuestas es la siguiente:

Mensaje	Respuesta
¿Cómo estás?	*Estoy bien*
¿Cuántos años tienes?	*1 año*
¿Eres el proceso hijo?	*Si*

Cualquier otra pregunta que se le realice al proceso hijo, retornará la frase: “No entiendo tu pregunta”. Las preguntas se realizarán por la entrada estándar hasta que se escriba la palabra “FIN”.

2. EJERCICIOS DE HILOS

2.a.1.Realizar un programa que cree 5 hilos y cada unos escriba “hola mundo”, y que además indique desde que hilo se imprime (los hilos se llaman por su número: 1,2,3,4 y 5. Luego haga que cada uno espere un tiempo proporcional a su identificador antes de imprimir el mensaje (el thread 1, un segundo, el 2, dos segundos, el 3, tres segundos,...). Lance los threads mediante la clase Thread y luego mediante el interfaz Runnable ¿Se advierte alguna diferencia?

2.a.2.Escriba un programa secuencial (sin hilos) que cree tres array de 1000000 de números, y realice la suma de los valores de cada uno, escribiendo los resultados. Después haga otra versión del programa donde esta tarea se realice entre tres threads, cada uno ocupado de un array. Mida el tiempo que emplea cada una de las versiones.

2.a.3.Ejecuta un programa que simule una carrera de coches, realizándolo en varios pasos:
Paso 1: hay 5 coches (hilos) que hacen una carrera de 10000 vueltas. Cada coche debe indicar cuándo acaba cada vuelta, y cuándo acaba su carrera.
Paso 2: El hilo principal ha de enviar un mensaje cuando han acabado todos los coches.
Paso 3: El hilo principal ha de informar de cuánto han tardado *todos* los coches en acabar la carrera.
Paso 4: Cada coche (hilo) ha de informar cuando acabe cuánto ha tardado en acabar *su* carrera.

- MEJORA 1 (usando monitores)

Paso 5: El hilo principal debe informar de la suma de los tiempos de cada uno de los coches
Paso 6: El hilo principal informa de quien ha ganado la carrera
Paso 7: El hilo principal informa del coche ganador y del ultimo en llegar

2.a.4.Escribe un programa que lance una carrera de 8 corredores, representado cada uno de ellos por un hilo:

- Cada corredor tendrá un número de dorsal desde el 1 hasta el 8 y tendrá que dar 20 vueltas (en un bucle for por ejemplo) para llegar a la meta.
- Cada vuelta se tarda 100 milisegundos.
- Se avisa de cuando han salido todos los corredores.
- Los corredores avisan por consola con su numero según vayan terminando.

- MEJORA 1:
  - Cuando acaben todos los corredores, se imprime un mensaje "Se acabó la carrera"

- MEJORA 2:
  - Cuando termina un corredor informa además de la posición en la que ha terminado.

Un ejemplo de ejecución podría ser el siguiente:

```
Carrera en transcurso...
Resultados:
Posición: 1 corredor 5
Posición: 2 corredor 3
Posición: 3 corredor 7
Posición: 4 corredor 1
Posición: 5 corredor 2
Posición: 6 corredor 6
Posición: 7 corredor 4
Posición: 8 corredor 8
```

- MEJORA 3:

- Sólo cuando hayan llegado los 8 corredores se mostrará el listado de resultados (se hace por lo tanto desde el método que lanza lo hilos)

2.a.5. Escribe un programa que simule una ejecución de un equipo de atletismo en una carrera de relevos. Se tiene por lo tanto 4 hilos (uno por corredor) que hacen una vuelta cada uno al estadio.
Cada vuelta se compone de 10 pasos y hay 100 milisegundos entre paso y paso
Tras arrancar el primer corredor, el segundo no arranca hasta que el primero termina, y el tercero no comienza hasta que termina el segundo, etc.
Se debe mostrar por pantalla cuando comienza cada relevo y cuando termina cada relevo

- MEJORA 1:
  - Se muestra al final el tiempo que han tardado los 4 relevos en cubrir la carrera
- MEJORA 2:
  - En la clase principal se deben lanzar los 4 hilos a la vez, de modo que aparezca un mensaje "Todos los hilos lanzados" y al final "Todos los hilos terminados".
- MEJORA 3:
  - Simular una carrera real de relevos con 4 equipos de 4 relevos cada uno. Se muestra al final el tiempo que han tardado cada equipo y quien ha sido el vencedor

2.a.6. Realizar un programa en el que 3 trenes salen de Salamanca hasta Cadiz. Cada uno hace 5 paradas, y en cada parada tarda un tiempo aleatorio entre 300 y 800 milisegundos. Cuando un tren llega a Cadiz, informa de que ha llegado y dice cuál ha sido la parada en la que más tiempo ha estado parado.
Cada tren informa tambien de cuanto ha tardado en total en llegar desde Salamanca a Cadiz
La clase principal que lanza los trenes informa del tiempo que han tardado todos los trenes en hacer el trayecto.

2.a.7. Realiza una aplicación que simule una carrera de motos (de hasta 4). Para cada moto se podrá configurar su velocidad y en la aplicación podremos configurar la distancia del circuito. Una vez lanzada la carrera se irá mostrando por pantalla el desarrollo de la misma (el avance de cada moto en el tiempo). Al final de la carrera se anunciará la moto ganadora y los demás se detendrán mostrando cuánta distancia han recorrido hasta ese momento

2.a.8. Realizar un programa en el que 4 cocineros hacen tartas. Cada cocinero tiene su nombre. Cada cocinero cocina 10 tartas, y tarda un tiempo aleatorio en hacer una tarta. Los cuatro cocineros empiezan a la vez a hacer tartas. Cada cocinero va informando de cuando acaba cada tarta y, cuando se acaban todas las tartas, el programa principal informa de cuánto ha tardado cada uno de los hilos en hacer su trabajo

2.b. EJERCICIOS NIVEL MEDIO

2.b.1. Realiza una aplicación que simule una carrera de motos, donde el número de motos se pide al usuario al comenzar el programa. Para cada moto se podrá configurar su velocidad y en la aplicación podremos configurar la distancia del circuito. Una vez lanzada la carrera se irá mostrando por pantalla el desarrollo de la misma (el avance de cada moto en el tiempo). Al final de la carrera se anunciará la moto ganadora y los demás se detendrán mostrando cuánta distancia han recorrido hasta ese momento

2.b.2. El siguiente programa rellena un array de gran tamaño con números aleatorios de tipo long y después muestra el tiempo empleado en buscar el mayor de ellos en milisegundos. Modifica el programa para que se realice la búsqueda del número máximo usando dos hilos. Calcula también el tiempo que tarda en hacerlo de esta otra manera. Implementa esta funcionalidad en el mismo código proporcionado.

```
import java.util.Random;
public class Main{
    private static final int NUM = 172000000;
    public static void main(String[] args)    {
        System.out.println("Rellenando aleatorios...");
        long[] numeros = new long[NUM];
        rellenaAleatorios(numeros);
        System.out.println("Array relleno. Empezando búsqueda sin hilos...");
        // Guardamos el tiempo de comienzo del proceso.
        long timeComienzo = System.currentTimeMillis();
        System.out.println("El número máximo es: " + numeroMaximo(numeros));
        System.out.println("Total milesegundos empleados SIN hilos: " +
                (System.currentTimeMillis() - timeComienzo));
        // Guardamos el tiempo de comienzo del proceso y ahora buscamos CON DOS HILOS:.
        timeComienzo = System.currentTimeMillis();
            ...
            ...
        System.out.println("Total milesegundos empleados usando 2 hilos: " +
                (System.currentTimeMillis() - timeComienzo));
    }
    private static void rellenaAleatorios(long[] num)    {
        Random rand = new Random();
        for(int i=0; i<num.length; i++)        {
            num[i] = rand.nextLong();
        }
    }
    private static long numeroMaximo(long[] numeros)    {
        long max = Long.MIN_VALUE;
        for (int i = 0; i < numeros.length; i++)        {
            if(numeros[i] > max)
            {
                max = numeros[i];
            }
        }
        return max;
    }
}
```

3. EJERCICIOS DE MONITORES SIMPLES (sin wait ni notify)

3.a.1.Escribir un programa consistente en la adivinación de un número entero por parte de dos jugadores (hilos) que se van turnando.

- El árbitro escribe al azar un número oculto a los jugadores mayor que 0 y menor que 500.
- El árbitro escoge al azar cuál será el primer jugador que empiece.
- Cada jugador dirá un número al azar al árbitro dentro del intervalo. En caso de adivinarse se acaba la partida, y en caso contrario se pasa el turno al otro jugador.
- Los jugadores no tienen ningún tipo de estrategia ya que dicen el número a adivinar al azar, eso sí, dentro del intervalo, pudiendo incluso repetir número en turnos anteriores.
- No hay número de intentos, pero al acabar hay que mostrar el número de intentos que el jugador ha necesitado para adivinar el número.

Ejemplo de ejecución:

```
Arbitro: Número escrito. Comienza la partida.
Arbitro: El turno es para el jugador B.
Jugador B: 248
Jugador A: 128
Jugador B: 63
Jugador A: 452
Jugador B: 8
Jugador A: 1
Jugador B: 207
Jugador A: 128
Jugador B: 49
Jugador A: 66
Jugador B: 331
Jugador A: 103
Jugador B: 243
……
Jugador A: 285
Arbitro: Jugador A acierta el número después de 28 intentos.
```

3.a.2. Queremos representar un banco. Tenemos varios clientes: Eva, Ana e Ines. El banco sólo tiene 2000 euros en metálico, y los clientes quieren sacar dinero constantemente debido a la crisis, y lo hacen desde un cajero automático. Sacan tanto como puedes y a todas las horas y todo lo rápido que pueden. Sacaran 200 euros cada vez. Gestionar el posible descubierto en la cuenta, teniendo en cuenta que el Ana es un cliente VIP y tiene prioridad por sacar dinero. Queremos saber cuánto dinero queda en el banco al final, y cuánto dinero ha sacado cada uno.

3.a.3.Desarrolle un programa en el que hay dos hilos.
El primer hilo hace 100 veces la acción de añadir un valor en un ArrayList de String (el valor es una cadena cualquiera de texto, siempre la misma). Espera 100 milisegundos entre cada acción de añadir.
El segundo hilo hace 100 veces la acción de quitar del mismo ArrayList el primer elemento. Espera 200 milisegundos entre cada acción de quitar.
Tanto uno como otro hilo informan tras cada acción de lo que han hecho (añadir o quitar) y de cuantos elementos quedan en el ArrayList
Probar y si todo va bien, al final debe informarse de que no hay elementos en el ArrayList
Luego probar a hacer que el hilo que pone espere 200 milisegundos y que el que quita espere 100... y veremos que falla. Investigar por qué falla, y entender que si no se usan otros mecanismos (que se ven en capítulos siguientes) no se puede ejecutar así el programa (probar a arreglar este programa cuando se conozca el uso de wait y notify)

3.a.4.Realizar un programa en el que contamos con una caja fuerte que tiene 50 lingotes de oro, y hay 3 ladrones que van sacando un número al azar de lingotes de la caja (un número entre1 y 5). Cada ladrón informa de los lingotes que va robando. Cuando ya no haya mas lingotes en la caja fuerte, todos los ladrones terminan y avisan de que han terminado

3.b. EJERCICIOS NIVEL MEDIO

3.b.1.El problema del zoo descontrolado

En un zoo hay varios cuidadores bastante descerebrados, que usan una jaula para sus siniestros juegos
La jaula tiene una capacidad limitada (que en el proyecto se guarda en una constante, pongamos inicialmente 10)
Los elementos que se pueden meter en la jaula son leones, monos y plátanos.
El número de cuidadores tambien es una constante en el proyecto (pongamos 3)
Cada cuidador tiene que meter tambien un número determinado de elementos en la jaula (otra constante del proyecto)
Los cuidadores meten continuamente elementos (escogidos al azar: un león, un mono o un plátano) y esperan un tiempo entre cada acción. Sin embargo, si un cuidador intenta meter algo y la jaula está llena, dicho cuidador termina su ejecución y se marcha a su casa.
Dentro de la jaula, se dan estas reglas de alimentación:
- los monos se comen los plátanos que haya
- los leones se comen a los monos que haya (pero no a los plátanos)

Tras la introducción de cualquier elemento se ha de informar de:
- quien mete el elemento
- que elemento mete
- contenido de la jaula tras meter el elemento y producirse las reglas de alimentación

3.b.2.Vamos a recibir una simulación de la fabula que cuenta la carrera entre la liebre y la tortuga. Habrá un thread que implementará la tortuga y otro la liebre. Para hacerlo más interesante la carrera será cuesta arriba por una pista resbaladiza, de modo que a veces podrán resbalar y retroceder algunas posiciones. Cada hilo se suspenderá durante un segundo y luego evaluará lo que ha pasado según unas probabilidades:

ANIMAL	SUCESO	PROBABILIDAD	ACCION A REALIZAR
LIEBRE	Duerme	20 %	No avanza ni rerocede
	Salto pequeño	20 %	Avanza 1 casilla
	Salto grande	30 %	Avanza 8 casillas
	Resbalón pequeño	10 %	Retrocede 2 casillas
	Resbalón grande	20 %	Retrocede 10 casillas
TORTUGA	Anda poco	50 %	Avanza 1 casilla
	Anda mucho	30 %	Avanza 4 casilla
	Resbala	20 %	Retrocede 5 casillas

La pista donde ambos corren tiene 70 casillas, de la 1 (salida) a la 70 (llegada).
Imprimir al comienzo de la carrera un mensaje que indica que se inicia la carrera.
Ahora los dos animales van a ir actuando, en un bucle. En cada actuación, cada animal:
- Calcular la probabilidad con random, de 1 a 100, y determinar con dicho número que ha va a hacer cada animal, y la nueva posición en la pista

- Si resbalan al principio vuelven a la 1, nunca por debajo.
- Tras cada segundo y después de calcular su nueva posición, se imprime una línea por cada animal, que contiene: el número de segundo, luego las casillas recorridas (se representan con "guiones bajos") y luego una letra T para la tortuga y una L para la liebre. Por ejemplo:

```
1_______T
1____L
2_________________T
2__________L
3__________T
4_______________L
```

- Después de imprimir las líneas se determina si alguno ha llegado a meta y ha ganado, imprimiendo un mensaje.
- Si ambos llegan a la vez declare un empate.

3.b.3. Sumando una matriz.

Se tiene una matriz de 20 filas y 100 columnas
Se ha de rellenar con números aleatorios entre 0 y 2
Se han de crear 20 hilos, que recorra cada uno una fila de la matriz, e irán contando cuantos ceros, unos y doses van encontrando, informando de cada numero que vayan analizando
Al final, se escribe la martriz entera y el porcentaje de ceros, unos y doses que hay en la matriz

4. EJERCICIOS PRODUCTOR-CONSUMIDOR (o de monitores con wait-notify)

4.a.1.Escribir un programa en el que dos usuarios escriben su nombre en una pizarra, 5 veces cada uno, pero han de hacerlo de modo alterno, de modo que se muestre en la pizarra

```
Ana
Eva
Ana
Eva
....
```

4.a.2.Hagamos un programa que inicialmente no se comporte como un productor-consumidor, y luego lo arreglaremos.

Un panadero va creando barras de pan. Tarda 100 milisegundos en hacer un pan. Cuando hace un pan, lo deposita en un cesto. Por otro lado, un Dependiente va recogiendo una barra de pan del cesto, y tarda 200 milisegundos en venderla.

Hacer un programa en el que el panadero construya 100 barras de pan, y el dependiente las venda. Cada vez que el panadero pone una barra en el cesto imprime un mensaje "he dejado una barra y hay x en el cesto", y cada vez que el dependiente coge un barra dice "he cogido una barra y quedan x en el cesto"

Si ejecutamos, veremos que es posible que en algún momento dado, ¡¡ haya un número negativo de panes en el cesto !! (si no se ve bien, probar a invertir los tiempos, que el panadero tarde 200 y el dependiente 100)

Arreglar el programa para que , por un lado, no pueda el dependiente sacar panes del cesto ,si no hay panes en el cesto, y que el panadero no pueda meter mas de 5 panes en el cesto, pues en el cesto en realidad solo van a caber 5 panes.

MEJORA 1: Probar estos cambios: hay 2 panaderos y 3 dependientes, y en el cesto caben 10 panes. Probar con varios tiempos diferentes para unos y otros.

4.a.3.Ejercicio Carpinteria. Se pretende sincronizar la fabricación en una línea de ensamblado de mesas.

Hay 4 fabricantes de patas, que las depositan en un almacén con un límite de capacidad de 10 patas Cuando se llena, los fabricantes dejan de producir patas hasta que haya hueco libre.

Hay 2 fabricantes de tableros, que depositan en un almacén de capacidad limitada de 6 tableros

Por último, hay dos ensambladores de mesas: cada uno coge cuatro patas del almacén de patas y un tablero del almacén de tableros y ensambla una mesa. No puede montar una mesa si no hay patas o tableros suficientes.

Se trata de escribir en Java un monitor que sincronice estos tres sistemas, de forma que la producción se detenga cuando se alcanza la capacidad máxima de algún deposito de almacenamiento (de patas o tableros independientemente) y que el sistema de ensamblaje no avance si le faltan piezas para hacer una nueva mesa.

Cada fabricante de patas fabrica una pata cada TIEMPO_PATA milisegundos (TIEMPO_PATA es una constante de la clase)

Cada fabricante de tableros fabrica una pata cada TIEMPO_TABLERO milisegundos (TIEMPO_TABLERO es una constante de la clase)

Cada montador de meses tarda TIEMPO_MESA milisegundos en montar una mesa (TIEMPO_TABLERO es una constante de la clase)

Cada fabricante de patas crea patas sin parar, los fabricantes de tableros crean sin parar y los ensambladores ensamblan sin parar

MEJORA 1: Cada fabricante de patas crea 20 patas, cada fabricante de tableros crea 10 tableros y los ensambladores terminan cuando terminan todos los fabricantes

4.a.4.Realizar un programa que simule la compra (consumidor) y la fabricación (productor) de coches. Debe cumplir estas normas:

Habra cinco tipos de compradores, y se identifica cada uno de ellos con un numero, de modo que el comprador numero 1 solo compre coches AUDI, el numero 2 FORD, el numero 3 SEAT, el numero 4 BMW y el numero 5 FIAT

Todos los compradores intentan continuamente comprar coches, si no existen coches de su marca, debe esperar a que haya alguno

Solo hay un fabricante, que fabrica continuamente coches de las 5 marcas, cada vez de una marca, elegida aleatoriamente. Tarda medio segundo en fabricar un coche.

El proceso continua hasta que el fabricante ha fabricado 20 coches..

- MEJORA 1: Al terminar todo, el programa indica cuantos coches ha comprado cada comprador

4.a.5.Realizar un programa que simule un concurso de comedores de tartas, donde deben cumplir estas normas:

- Participan 3 pasteleros y 6 "zampones" de tartas
- Cada pastelero hace una tarta distinta (fresa, limón, queso) que va dejando en una mesa
- En la mesa no caben mas de 5 tartas
- Cada "zampón" solo come tarta de un tipo, y hay dos "zampones" de cada cosa (dos "zampones de tartas de limón", dos de tarta de fresa y dos de tarta de queso). Para comer, coge una tarta de la mesa y la devora, tardando entre 300 y 600 milisegundos en acabar. Evidentemente, cada zampón solo puede coger tartas de lo que él come, y no coge nada mientras no haya de lo suyo.
- Cada comedor tiene su nombre
- Cada pastelero tarda entre 200 y 500 milisegundos en hacer una tarta
- Cada pastelero crea un numero al azar de tartas, entre 3 y 8
- Al final, hay que decir cuantas tartas ha comido cada “zampón"
- Cada zampón zampa sin parar, y los pasteleros cocinan sin parar

- MEJORA 1:
  - Cada pastelero tarda entre 100 y 300 milisegundos en hacer una tarta
  - Cada "zampón" tarda entre 300 y 600 milisegundos en comer
  - Cada pastelero no puede crear mas tartas de lo suyo si hay 2 o más tartas lo suyo sin comer en la mesa

- MEJORA 2: Cada pastelero cocina 10 tartas, y acaba (se retira a su casa). Cada zampón termina cuando termina su pastelero correspondiente (los zampones de limón acaban cuando se va a su casa el pastelero de limón, e igual los otros zampones con sus pasteleros correspondientes)

- MEJORA 3: Solo existe una silla donde los "zampones" pueden comer, esto es, no puede haber mas de un "zampón" comiendo simultáneamente (se puede hacer con semáforos o atributos sincronizados)

4.a.6.Escriba un monitor en java que controle el acceso a un parking de coches. El parking tiene un número de plazas (digamos 10) y dispone de dos accesos de entrada, Este y Oeste. Se introducen 20 coches en el parking por cada puerta.

Si el parking no está lleno, se admiten entradas por ambos accesos libremente.

Si el parking está lleno, los coches deben esperar a que haya plazas.

Existen dos vías de salida, Norte y Sur. Cuando un coche abandona el parking, se considera irrelevante el acceso que usa para salir. Cada coche esta dentro del garaje entre 0.5 y 1 segundos.

- MEJORA 1:

- El monitor debe alternar los accesos de los coches por las entradas Este y Oeste.

4.a.7. Un transbordador permite pasar coches de un lado de un río al otro. El transbordador tiene una cabida de 10 coches y espera a estar lleno para cruzar el río. Cuando ha cruzado y descargado los coches, vuelve vacío. Simulamos el tiempo de traslado con una espera de 2 segundos. Los coches que han cruzado no vuelven.
Se trata de implementar este problema con procesos (coches), resolviendo la concurrencia con el empleo de un monitor (el transbordador). Los coches que se crean y llegan al puerto de embarque del transbordador serán 100. Los 100 coches llegan a la vez a la orilla, pero no suben al transbordador mas que los que puedan en cada caso. El proceso termina cuando los 100 coches hayan cruzado el rio. Se debe informar por consola de los coches cuando llegan al rio (antes de subir al transbordador) y los coches que alcanzan la otra orilla

4.a.8. Sea un cruce de calles, por el que circulan coches de oeste a este y de norte a sur. Para regular el tráfico hay dos semáforos, uno en la entrada oeste y otro en la entrada norte, y dos sensores que se activan cuando llega un coche a la entrada correspondiente. También hay sensores que indican la salida del cruce.
Se desea desarrollar un monitor en Java que simule la gestión de los semáforos de la siguiente forma:
- Los coches se modelan como hebras (threads) que invocan un método llegaNorte() o llegaOeste() cuando llegan al cruce. Eligen uno de los dos métodos al azar.
- Si el semáforo correspondiente está en verde, el coche pasa inmediatamente. Si está en rojo, espera hasta que esté verde.
- Los coches tardan un cierto tiempo en pasar el cruce. Al salir invocan un método sale() en el monitor.
- Una hebra de control llama a un método cambiaSemáforos() cada cierto tiempo para cambiar la configuración de los semáforos.
El monitor podría tener una pinta como esto...:

```
class GestionCruce { ...

      ... llegaNorte() {...} // lo invoca un coche que llega por el N

      ... llegaOeste() {...} // lo invoca un coche que llega por el W

      ... sale() {...} // lo invoca un coche que sale del cruce

      ... cambiaSemáforos() // lo invoca la hebra de control
}
```

4.a.9. Hilos alternativos
Se tiene un sistema con dos hilos (H1 y H2) que acceden continuamente a un recurso compartido, que se debe usar con exclusión mutua. Para que el sistema funcione correctamente las hebras tienen que acceder alternativamente, según el siguiente patrón, que se debe repetir cíclicamente: H1, H2, H1, H2, ..
Se pide desarrollar el monitor (GestorCiclosAcceso) para sincronizar los hilos según el comportamiento descrito. El monitor puede por ejemplo proporcionar las siguientes operaciones:

```
... void accederH1(): Este método lo invoca la hebra H1 para solicitar acceso al recurso
compartido.
... void accederH2(): Este método lo invoca la hebra H2 para solicitar acceso al recurso
compartido.
... void liberar(): Este método lo invocan las hebras para liberar el recurso compartido.
```

4.a.10. La discoteca supersticiosa
Imaginemos un programa que es muy "supersticioso", un programa donde puede haber cualquier número de hebras, siempre que no sean trece. Se pide realizar una clase sincronizada que realice tal protocolo de sincronización.

Imaginad un programa que lleve el control de acceso a una discoteca. Cuando un usuario llega a la discoteca, entra en ella, esta un tiempo aleatorio bailando, y abandona la discoteca. En la discoteca no puede haber mas de 20 personas, así que el control de acceso debe prohibir entrar a más gente si ya hay 20. Además, no puede haber exactamente 13 personas en la discoteca, o hay 12 o menos, o 14 o mas. Para ello, el control de acceso debe controlar que :

- Si llega el cliente nº 13 debe esperar a otro cliente que quiera entrar, o a que alguien de dentro quiera salir.
- Lo mismo pasaría si se desea salir, dejando 13 clientes dentro: deben salir 2 o compensar con uno que desee entrar.

Haced en el programa que se creen 100 clientes, y que para llegar a la discoteca tarden un tiempo aleatorio entre 100 y 10000 milisegundos. Controlar tambien que el tiempo que bailan este aleatoriamente entre 1000 y 6000 miilsegundos. Ejecutar el programa con estos valores, y posteriormente, cambiando estos limites para ver varios comportamientos.

4.a.11. Caníbales y Misioneros, versión Siglo XIX (Cocineros ansiosos). Una tribu de caníbales comparte una olla en la que se meten misioneros. El cocinero cocina misioneros, y los va metiendo en la olla (tarda un tiempo x en preparar al cocinero antes de meterlo en la olla) y se pone a dormir. Cuando un caníbal llega a comer, coge directamente un misionero de la olla, a no ser que esté vacía. Si la olla está vacía, el caníbal debe esperar a que haya algún misionero. En la olla no caben más de 3 misioneros, por lo que el cocinero no puede meter más misioneros si la olla está llena. Implementar la solución teniendo en cuenta que:

- No se debe producir interbloqueo.
- Los caníbales podrán comer siempre que haya comida en la olla.
- Iniciar el ejercicio con 1 cocinero y 20 salvajes que llegan a la vez a comer a la aldea
- No hay límite de misioneros, el cocinero tiene tantos como desee cocinar y puede meter en la olla tantos como quiera
- El cocinero cocina 20 misioneros
- Cada caníbal come sólo un misionero. Cuando acaba de comer, se ponen a hacer la digestión y ya no participan más en el ejercicio.
- Cuando todos los caníbales acaban de comer, y el cocinero de cocinar, se emite un mensaje que diga “Todos los caníbales han comido”

4.a.12. Los fumadores. En torno a una mesa hay cuatro personas: tres fumadores y un proveedor. Cada fumador fuma cigarrillos uno detrás de otro. Para poder fumarse un cigarrillo se necesitan tres ingredientes: papel, tabaco y cerillas. Uno de los fumadores tiene papel, el otro tabaco y el otro cerillas. El proveedor tiene una cantidad infinita de los tres ingredientes.
Inicialmente, el proveedor coloca dos de los ingredientes en la mesa, escogidos al azar. El fumador que tiene el ingrediente que falta toma lo que hay en la mesa, lía un cigarrillo y se lo fuma (tarda unos milisegundos en fumárselo). Cuando termina, se lo indica al proveedor. El proveedor entonces coloca otros dos ingredientes y el ciclo se repite una y otra vez.
Se pide construir un programa compuesto por cuatro hilos (tres fumadores y el proveedor) que estén correctamente sincronizados.

MEJORA 1: Variar el comportamiento de los fumadores: Cuando un fumador toma lo que hay en la mesa, y antes de ponerse a fumar, avisa ya al proveedor para que ponga dos nuevos ingredientes en la mesa. De esta forma, mientras un fumador está fumando, se da la posibilidad de que otro fumador pueda tomar la pareja de ingredientes que hay en la mesa y así lleguemos a tener varios fumadores al mismo tiempo. (Aunque esto no ocurrirá si el proveedor coloca justo el par de ingredientes que no le sirven a los fumadores en espera, pero eso depende del azar).

4.a.13. Escribir un esquema tradicional de un productor/consumidor, en este caso, tiene una clase Elementos que permita almacenar en forma de pila y sin repetir un determinado número de elementos positivos de tipo int. El número de elementos máximo a almacenar será especificado en el constructor.

- El método *sacarElemento* extrae un elemento y lo devuelve. Devolverá -1 si la estructura está vacía.
- El método *ponerElemento* añade un elemento si hay espacio, no existe previamente, y bien es positivo o bien es 0. Devuelve false si no lo ha podido añadir, true en caso contrario.

Ejemplo de código inicial y ejecución:

```
public class Main
{
   private static final int NUM = 4;
   public static void main(String[] args) throws InterruptedException
   {
      Elementos elementos = new Elementos(NUM);
      System.out.println(elementos.sacarElemento()); // Vacía
      System.out.println(elementos.ponerElemento(0)); // Añade 0
      System.out.println(elementos.ponerElemento(0)); // Repetido
      System.out.println(elementos.sacarElemento()); // Extrae el 0
      System.out.println(elementos.ponerElemento(1)); // Añade 1
      System.out.println(elementos.ponerElemento(2)); // Añade 2
      System.out.println(elementos.ponerElemento(3)); // Añade 3
      System.out.println(elementos.ponerElemento(3)); // Repetido
      System.out.println(elementos.ponerElemento(4)); // Añade 4
      System.out.println(elementos.ponerElemento(5)); // No cabe
      System.out.println(elementos.sacarElemento()); // Extrae 4
   }
}
```

Salida por consola:

```
-1
true
false
0 true
true
true
false
true
false
4
```

Una vez comprobado el correcto funcionamiento de la estructura, crea dos hilos dedicados a hacer cada uno de ellos una tarea distinta sobre una instancia de la clase Elementos:

- threadPonerElementos : Añadirá, indefinidamente, enteros aleatorios.
- threadSacarElementos : Extraerá, indefinidamente, los enteros almacenados.

Para poder ver que está sucediendo, añade mensajes por consola en el código:

- Extraído el elemento X.
- Sin elementos.
- Añadido el elemento X.
- Elemento X ya existe.

4.a.14. Submarinos versión 1, con Productor-Consumidor: Estamos en la segunda guerra mundial, y los submarinos alemanes envían mensajes a su centro de control indicando sus próximos movimientos.

Los submarinos envían mensajes ficticios y validos aleatoriamente. Todos los mensajes (ficticios y reales) se extraen de este array de mensajes:

```
String [] mensajes = {"vamos al norte", "vamos al sur","yendo al sur","yendo al norte","mas
al sur","giro al sur","no nos movemos", "sin avanzar"};
```

Hay tres submarinos alemanes enviando mensajes al centro de control, un mensaje cada 200 milisegundos, mensajes que extraen aleatoriamente del array de mensajes.

El centro de control tiene a dos controladores trabajando. Ambos controladores tienen un buzón de entrada (común a todos los submarinos) en el que solo se pueden almacenar 5 mensajes.

Cada controlador procesa un mensaje cada segundo.

Cuando el controlador recibe un mensaje, si es ficticio lo añade a una lista, y si es verdadero, lo añade a otra lista distinta (ambas listas son comunes a los dos controladores), y después, elimina el mensaje

del buzón de entrada. Los controladores saben que un mensaje es cierto porque contiene la letra "m", son falsos los que no la contienen.
Hacer un proceso productor-consumidor que haga que los 3 submarinos envíen 20 mensajes cada uno. Los controladores informan por consola de lo que reciben y si es cierto o no.
Al final del proceso, se muestran por consola las dos listas, la de mensajes correctos y la de los mensajes incorrectos.

- MEJORA 1: solo hay dos controladores, y uno de ellos solo procesa mensajes que empiezan por "y", mientras que el otro solo procesa mensajes que no empiecen por "y"
- MEJORA 2: del número de submarinos no es fijo, se solicita al usuario al comenzar el programa
- MEJORA 3: Los submarinos envían los mensajes codificados con un algoritmo AES (simétrico), que los controladores deben desencriptar

4.a.15. Escribir un esquema Productor-Consumidor con monitor en modo LIFO. Un productor mete mensajes en un buffer (un array de 4 elementos), y un consumidor extraen los mensajes del buffer, de modo LIFO.
El productor coge los mensajes aleatoriamente de un array de Strings que contiene 10 palabras, una de ellas "fin".
Cuando el productor envia un mensaje "fin", deja de enviar mensajes.
Cuando el consumidor encuentra un mensaje "fin", debe vaciar el buffer y acabar .

4.a.16. La Gymkhana Interruptus. Se desea simular la realización de una Gymkhana. La reglas a tener en cuenta son las siguientes:

- La Gymkhana consta de 3 fases. Y cada fase consta de 2 pruebas.
- Contará con 3 participantes y un árbitro.
- Para simular el tiempo de realización de cada prueba, cada participante (hilos consumidores) hará un sleep de tiempo aleatorio entre 0 y 1 segundo.
- Antes de iniciar la prueba, todos los participantes han de esperar (wait()) a que el arbitro (hilo productor) les de la señal de salida (notifyall()..
- Al finalizar la fase 1 y la fase 2, todos los participantes esperarán a que un árbitro les de la señal para continuar a la siguiente fase.
- Al finalizar la fase 3 no se detendrán los participantes y se finalizará el programa, momento en que se mostrará, una vez que hayan finalizado todos, el puesto que ha ocupado en cada prueba cada participante.
- Cada fase tienen dos pruebas que se realizarán secuencialmente, sin parar entre una y otra prueba
- Para simular la obtención del lugar en el que ha finalizado cada prueba, cada participante llamará a un método syncronized tras el tiempo aleatorio. Ese método servirá para que cada participante obtenga la posición en la que ha entrado en ese método, memorizarla para imprimirse al final del programa, y aumentar la posición en 1 para que pueda recogerla el siguiente participante.
- Para simular la señal del árbitro, este ejecutará un notifyall() a los 2 segundos de empezar el programa (para lanzar a todos los participantes), tras ello, a los 5 segundos para lanzar la segunda fase, y tras ello, a los 5 segundos para lanzar la tercera fase.

4.a.17. Escribir un programa en que dos hilos escriben "PING" y "PONG" (un hilo escribe "PING" y el otro "PONG"). Pero los hilos han de escribir alternativamente, ninguno puede escribir dos veces seguidas su mensaje sin que el otro haya escrito lo suyo.

- MEJORA 1: Comienza siempre escribiendo el hilo que dice "PING"

4.a.18.Realiza una aplicación que simule la persecución de dos aviones. Considerando los parámetros altura (un valor entre 100 y 1000) y dirección (norte, sur, este, oeste), el avión perseguido puede trazar la ruta que quiera y cambiar su dirección y altura y el avión perseguidor deberá modificar esos parámetros en la misma medida en que lo haga el primero.
Puedes solicitar por consola cambiar los parámetros del avión perseguido o bien preparar un recorrido ya definido, o hacer que el avión perseguido cambie sus parámetros al azar cada x tiempo.
Lanzar la aplicación (que no termina nunca) para ver como lo recorren ambos objetos (mostrando en cada instante los parámetros de cada uno)

- MEJORA 1:

  El segundo avión solo cambia su trayectoria si cambia la dirección del primer avión (no hace nada si el primer avión solo cambia la altura), en cuyo momento se pone en la dirección del primer avión y a una altura de 10 metros por debajo

4.a.19.Un hilo principal va creando 5 hilos
1.-El hilo principal rellena un array con 5 números.
2.- Cada hilo coge un numero del array, cada uno un numero distinto, claro, vaciando el array, y escribe su numero por pantalla.
El hilo principal repite 3 veces el ciclo compuesto por los pasos 1 y 1, con 2 segundos de espera entre cada ciclo

4.a.20. Adivinando números.. Se va a simular un juego para adivinar un número. Se crearán 2 hilos, siendo estos los jugadores. Habrá un árbitro que genere de forma aleatoria el número a adivinar (de 1 a 10), comprobará la jugada del jugador y averiguará a qué jugador le toca jugar.
Se tienen 3 clases. Se recomienda implementar la siguiente estructura:

- Árbitro:
  - Determina el nº a adivinar o Comprueba las jugadas lanzadas por los jugadores y decide a quien le toca. o Si un jugador acierta el nº generado, el árbitro dará por terminado el juego.
- Jugador:
  - Cada uno tendrá un ID numérico para representarse.
  - Ambos compartirán a un mismo árbitro.
  - Cada uno de ellos va lanzando jugadas y se las pasa al árbitro para que este compruebe si el jugador acierta o no.
- Main:
  - Clase principal desde la cual crear el árbitro y los jugadores.

Asegurarse de que ningún jugador queda en “espera”.
El juego ha de funcionar de forma sincronizada y ordenada. Ejemplo de salida:

```
Número a adivinar : el 3
3 Jugador 1 juega: 9.
Fallo.
Turno de Jugador 2.
Jugador 2 juega 7.
Fallo.
Turno de Jugador 1.
Jugador 1 juega: 3 ¡Jugador 1 gana!
```

4.a.21.La discoteca supersticiosa

En un programa existe una zona crítica especial, llamémosla “supersticiosa”, en la que puede haber cualquier número de hebras, siempre que no sean trece. Se pide realizar una clase sincronizada que realice tal protocolo de sincronización.

Como el enunciado se presta a cierta ambigüedad en la interpretación de qué significa que “no hayan 13 threads en la zona crítica”, aplicaremos la interpretación del modelo que llamaremos “discoteca”, donde:

- Si llega el cliente nº 13 debe esperar a otro cliente que quiera entrar, o a que alguien de dentro quiera salir.
- Lo mismo pasaría si se desea salir, dejando 13 clientes dentro: deben salir 2 o compensar con uno que desee entrar..

4.a.22.El taller de costura

Tenemos un taller de costura dedicado a hacer jerséis. En su interior tenemos a seis personas trabajando:

- Una persona está continuamente fabricando mangas rojas, y otra fabricando mangas amarillas, que van depositando en un cesto (común para las dos). El cesto tiene una capacidad limitada: cuando se llena, las dos costureras de mangas dejan de coser más mangas hasta que hay hueco libre.
- Otra persona está continuamente fabricando cuerpos de jerseys rojos, y otra cuerpos de jerseys amarillos. Ambas también depositan los cuerpos en su correspondiente cesta de capacidad limitada.
- Una quinta persona se encarga continuamente de ensamblar jerséis rojos, y otra más de ensamblar jersey amarillos. Para ello, cogen en cada caso dos mangas de la cesta de mangas y un cuerpo de la cesta de cuerpos (pero solo los del color que les corresponde: la que ensambla jersey rojos, solo coge mangas rojas y cuerpos rojos, y la otra, lo mismo con los amarillos).

Escribir aun programa que sincronice a estas seis personas, de forma que las cuatro primeras personas no avancen si su cesta está llena, y que las dos ultimas no avancen si le faltan piezas para hacer un nuevo jersey.

Se supone que las capacidades de las dos cestas son constantes y distintas (supongan, p.ej. que son dos constantes enteras "NumMaxMangas" y "NumMaxCuerpos").

4.a.23. La biblioteca con los Cids en Vasco

Varios lectores vascos acuden diariamente a una biblioteca a consultar alguna de las 10 únicos ejemplares del Cid en vasco que hay en la biblioteca.

Cada ejemplar esta numerado del 1 al 10

Cuando un lector quiere solicitar un libro, debe elegir qué numero de ejemplar quiere, e intentar cogerlo para leerlo.

Puede coger dicho ejemplar si:

no esta siendo leído en ese momento por otro lector
hay menos de 3 lectores leyendo otros ejemplares en ese momento.
Si no consigue coger un ejemplar, espera hasta que pueda leer.

Si consigue coger el libro para leerlo, pasa un tiempo entre 500 y 1000 milisegundos leyéndolo, y luego lo devuelve, saliendo de la biblioteca hacia casa.

Hacer un programa que exprese la rutina diaria de la biblioteca, indicando por cada lector que esta haciendo... cuando intenta coger un libro (y que libro es), cuando lo devuelve, etc...

MEJORA 1:

Un lector puede coger dicho ejemplar si:

no esta siendo leído en ese momento por otro lector
hay menos de 3 lectores en ese momento leyendo "*otros ejemplares que tenga un número de ejemplar menor del que se solicita*"

MEJORA 2:

Por política de mantenimiento de calidad de los productos, no se puede estar leyendo material mas de 5 segundos al día. Por lo tanto, cuando un lector intente coger un ejemplar, si ya han consumido los 5 seguros diarios de lectura entre todos los lectores, nop puede coger nada y ha de irse a casa

4.b. EJERCICIOS NIVEL MEDIO

4.b.1. Productor Generoso (. Tenemos un proceso productor y 3 procesos consumidores que comparten un contenedor de mensajes. El contenedor tiene capacidad para 5 mensajes. EL productor va metiendo, poco a poco, 30 mensajes en el contenedor. Cada mensaje va destinado a todos los consumidores: esto es, el mensaje no desaparece del contenedor hasta que ha sido leído por los 3 consumidores. Cada consumidor va leyendo elementos cada x milisegundos (al azar entre 100 y 300), y cada vez que lee, lee uno de los mensajes que no haya leído aun, o si los ha leído todos (o no hay mensajes para leer), queda en espera de nuevos mensajes. Como los consumidores acuden a coger mensajes a su propio ritmo, puede haber consumidores mucho más adelantados que otros.

4.b.2. Se consideran dos recursos denominados r1 y r2. Del recurso r1 existen N1 ejemplares y del recurso r2 existen N2 ejemplares. Escribir un monitor que gestione la asignación de los recursos a los procesos de usuario, suponiendo que cada proceso puede pedir:

- Un ejemplar del recurso r1.
- Un ejemplar del recurso r2.
- Un ejemplar del recurso r1 y otro del r2.

Un recurso no será asignado a un proceso que demande un ejemplar de r1 o un ejemplar de r2 hasta que al menos un ejemplar de dicho recurso quede libre.
Se dará prioridad a los procesos que demanden un ejemplar de ambos recursos.

4.b.3.Escribir un programa que incluya una barrera. Una barrera es un punto de sincronización entre varios threads. Se caracteriza porque los threads que van llegando a ella esperan hasta que llega el último. Por ejemplo, se puede sincronizar N threads y hasta que el último no llegue los demás no deben poder continuar. Pruébela con el código del primer problema, es decir lancé unos pocos threads (cuatro es razonable), cada uno que espere un tiempo proporcional a su identificador, que luego imprima un mensaje que lo identifique y que luego espere en la barrera por los otros threads. Introduzca este esquema en un bucle para ver cómo evoluciona con el tiempo.

4.b.4.ChanchuBank y los politicorruptos.
Realizar un programa tipo productor-consumidor, que simule un Banco, cuyo banquero que hace de productor, y varios Políticos-Corruptos hacen de consumidores.
En el Banco hay inicialmente una caja B con 30.000.000 de euros.
El banquero coge de la caja B una cantidad al azar (entre 1.000.000 y 5.000.000 de euros) y la pone en la ventanilla del banco.
Todos los politicorruptos están esperando en la sala del banco, y solo uno de ellos cogerá el dinero de la ventanilla, y volverá a quedarse esperando que saquen más pasta.
El banquero no sacara más dinero a la ventanilla hasta que algún politicorrupto la vacíe.
Cuando se acaba el dinero de la caja, todos los politicorruptos se van a robar a otro banco, pero antes, informan por consola de cuánta pasta se han conseguido.
Cuidado que el banco no puede quedarse con números negativos…….

4.b.5. Tenemos una base de datos a la que acceden dos tipos de procesos: lectores y escritores. Los procesos lectores consultan registros de la base de datos, mientras que los procesos escritores modifican esos registros. La base de datos se puede acceder de forma concurrente, pero controlando que no se corrompa la información. Para ello se tienen que cumplir estas reglas:

- Puede haber varios lectores trabajando concurrentemente.
- Si un escritor intenta acceder a la BD mientras hay lectores trabajando, el escritor debe esperar a que la BD quede libre.
- Mientras un escritor está trabajando con la BD, no puede haber ningún otro proceso trabajando (ni lector ni escritor).

Se puede hacer dos versiones distintas del programa, según cómo se dé prioridad a lectores o escritores cuando la BD queda libre:

- Primera versión: un proceso lector puede entrar en cuanto sea posible.
- Segunda versión: en cuanto haya un escritor esperando a entrar en la BD, los lectores que vayan llegando nuevos deben esperar, incluso si solo hay lectores trabajando en la BD (se pueden usar semáforos tambien en este caso)

4.b.6. Caníbales y Misioneros, versión Siglo XX (El cocinero vago) . Una tribu de caníbales comparte una olla en la que se meten misioneros. El cocinero cocina X misioneros (X es un valor entre 1 y 4), los mete en la olla y se pone a dormir. Cuando un caníbal llega a comer, coge directamente un misionero de la olla, a no ser que esté vacía. Si la olla está vacía, el caníbal despertará al cocinero y éste llena la olla con otros X misioneros, y vuelve a dormir. Implementar la solución teniendo en cuenta que:

- No se debe producir interbloqueo.
- Los caníbales podrán comer siempre que haya comida en la olla. Cuando acaban de comer, se ponen a hacer la digestión y ya no participan más en el ejercicio.
- Solamente se despertará al cocinero cuando la olla esté vacía. Esto es importante, solo se le despierta si no hay nada que comer, si no, se le deja dormir.
- Iniciar el ejercicio con 1 cocinero y 20 salvajes que llegan a la vez a comer a la aldea
- No hay límite de misioneros, el cocinero tiene tantos como desee cocinar y puede meter en la olla tantos como quiera
- El cocinero cocina sin parar, pero
- Cuando todos los caníbales acaban de comer, se emite un mensaje que diga “Todos los caníbales han comido”

4.b.7. Caníbales y misioneros, versión Siglo XXI (La canoa sensible).

En el África profunda, en el siglo XXI, aún viven junto a un río una tribu de caníbales y un grupo de misioneros. El río se puede cruzar mediante una canoa, con capacidad para tres personas. Como las aguas son bravas, la canoa siempre tiene que transportar carga completa, por lo que no cruce si no hay tres personas dentro. Pero si se sientan dos misioneros y un caníbal, éstos le comen el tarro al caníbal, hasta el punto de que el caníbal se arroja al agua: hay que evitar esta situación. Las demás combinaciones no dan problemas.

El sistema debe lanzar varios hilos, uno por cada persona. Escriba dos procedimientos: LlegaMisionero() y LlegaCaníbal(), para cuando cada persona llega a la canoa. Estos procedimientos dejan al hilo esperando hasta que la canoa se encuentre llena y segura. Cuando esto ocurra, el procedimiento retorna y se considera que la canoa vuelve a quedar vacía.

4.b.8. Problemas del baño mixto. Ante la carencia de baños en el Ministerio de Meones, el Ministro ha decidido que los pocos baños sanos sean de uso mixto: pueden entrar tanto hombres como mujeres, pero con la condición de que simultáneamente sólo pueda haber personas de un único sexo. Implemente la gestión de uno de estos baños (cada persona se simula como un hilo que entra y sale del baño). Crear por lo tanto 50 hilos (al azar, algunos hombres y otros mujeres) y mandarlos al baño (el baño inicialmente no tiene limite de capacidad)

- MEJORA 1: Controlar que el baño tenga una capacidad limitada.
- MEJORA 2: Controlar que no se pueda producir inanición.

4.b.9. Lectores y escritores

Tenemos una base de datos a la que acceden dos tipos de procesos: lectores y escritores. Los procesos lectores consultan registros de la base de datos, mientras que los procesos escritores modifican esos registros. La base de datos se puede acceder de forma concurrente, pero controlando que no se corrompa la información. Para ello se tienen que cumplir estas reglas:
• Puede haber varios lectores trabajando concurrentemente.
• Si un escritor intenta acceder a la BD mientras hay lectores trabajando, el escritor debe esperar a que la BD quede libre.
• Mientras un escritor está trabajando con la BD, no puede haber ningún otro proceso trabajando (ni lector ni escritor).
Hay dos planteamientos para el programa, según cómo se dé prioridad a lectores o escritores cuando la BD queda libre: según el primer problema, un proceso lector puede entrar en cuanto sea posible. Según el segundo problema de los lectores escritores, en cuanto haya un escritor esperando a entrar en la BD, los lectores que vayan llegando después deben esperar, incluso si solo hay lectores trabajando en ese momento en la BD.
Proponer soluciones a ambas versiones.

4.b.10. El tráfico en el puente estrecho

Construya un sistema para gestionar el tráfico sobre un puente estrecho. El puente lo pueden atravesar vehículos desde ambos extremos. Como el puente tiene un solo carril, en un momento dado sólo puede haber vehículos cruzándolo en un único sentido. El puente tiene una capacidad de carga limitada: si hay más de tres vehículos, se hunde. Implemente una solución en la que cada vehículo sea un hilo.
Pista: el algoritmo que sigue cada hilo-vehículo debe ser parecido a esto:

```
void run () {
  llegar (sentido)
  cruzar (sentido)
  salir (sentido)
}
```

El parámetro sentido indica en qué sentido se cruza el puente (o sea, que puede tener dos valores, por ejemplo izquierda y derecha).
Crear 20 vehículos y hacerlos cruzar (no deben llegar al mismo tiempo al puente, tardarán en llegar un tiempo aleatorio, por ejemplo, entre 200 y 4000 milisegundos. Si llegan todos al mismo tiempo, es posible que solo entren todos a la vez por un lado y luego todos a la vez por el otro...)
El tiempo que tardan en cruzar es siempre el mismo para todos los vehículos (200 milisegundos)
Garantice que no ocurren colisiones y que el puente no se hunde.
Además se debe garantizar que no se produzcan esperas indefinidas.

4.c. EJERCICIOS NIVEL ALTO

4.c.1. Un deposito de agua es llenado a un ritmo de 10 litros por segundo. Cuando ese depósito llega a los 900 litros, se activa un proceso de vaciado a un ritmo de 5 litros por segundo. Cuando el depósito llega a los 1000 litros, el proceso de llenado se para y se aumenta el de vaciado a 10 litros por segundo. Cuando el depósito llega a los 100 litros de vaciado, se activa un proceso de llenado a un ritmo de 5 litros por segundo. Cuando el depósito llega a los 0 litros, el proceso de vaciado se para y se aumenta el de llenado a 10 litros por segundo. Simula con monitores dichos procesos y ve representando en pantalla las acciones que se van tomando.

4.c.2. La Gincana

Se plantea un ejercicio para simular una gincana, con estas reglas:

- Habrá 5 participantes, y cada participante en la gincana pasa por 3 fases.
- Al empezar cada una de las 3 fases, los participantes deben esperar que un arbitro (otro hilo) les de la señal para comenzar.

- Al finalizar la fase 3, no se espera a nada mas, el participante simplemente termina.
- Las fases 1 y 2 constan a su vez de dos pruebas, que se hacen seguidas, sin parar entre ellas. La fase 3 consta de una única prueba. Por lo tanto, en total, cada participante realiza 5 pruebas.
- El tiempo de cada uno tarda en cada prueba se simula aleatoriamente entre 1 y 3 segundos
- Se debe controlar el orden de finalización de todos los participantes en cada una de las 5 pruebas, informando de las posiciones de los participantes al fin de cada prueba.
- Para simular las señales de salida del arbitro para cada fase, éste efectuará un aviso (notifyAll, por ejemplo) en los segundos 5, 15 y 25 desde el inicio del programa

Se pueden usar monitores, semáforos y cualesquiera otras herramientas de control de procesos.

5. EJERCICIOS DE SEMÁFOROS

5.a.1.Ejercicio Barbería versión 1. En una barbería hay un peluquero, que tiene una silla para afeitar a los clientes.

Para hacer que los clientes no lleguen todos a la vez, cada cliente esta "paseando" un tiempo al azar antes de ir a la barbería (entre 100 y 1000 milisegundos, al azar).
Se crean 20 clientes para ir a la barbería.
Si llegan clientes se sientan si la silla de afeitar esta libre, si no, esperan en la calle a que haya sitio libre.
Cada cliente tarda en pelarse entre 300 y 1000 milisegundos, al azar.
Hacer que se creen tantos clientes como un valor que se pida al usuario al inicio del programa.
Definir el semáforo necesario y su inicialización.
En el programa se debe ir indicando por consola las acciones de los clientes: cuándo llegan, cuándo comienzan a pelarse, cuándo se van...

5.a.2.Ejercicio Parking . Tenemos un centro comercial que tiene un parking con 10 plazas
Hay 40 coches circulando por la ciudad que van a intentar entrar al parking.
Solo existe una entrada al parking.
Cada hilo comienza circulando un tiempo aleatorio (hasta que llegan al parking) y después intentará entrar al parking, si hay plaza, o esperar si no hay plaza libre.
Una vez dentro del parking el coche debe estar un tiempo aleatorio (entre 1 y 3 segundos).
Se debe informar por consola de las actividades que hace cada coche.

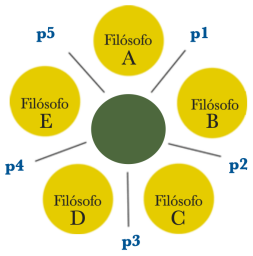
- MEJORA 1: Los coches entran en el parking en el mismo orden en que llegan al mismo. Esto es, deben hacer cola según llegan y entran en el mismo orden en que llegan a la cola
- MEJORA 2 Cada coche gasta una pasta en el parking, 0.2 céntimos de euro por milisegundo que esta en el parking. Decir al final del proceso cuánto ha ganado el parking con el paso de todos los coches
- MEJORA 3: Mostrar cuántos coches hay en la cola del parking en cada momento
- MEJORA 4: Los coches, cuando llegan al parking, si no hay plazas libres, no intentan entrar en el parking, se piran a casa (lo indican por consola)
- MEJORA 5: Los coches, cuando llegan al parking, si en la cola hay esperando mas de 3 coches, no se quedan en el parking, se piran a casa (lo indican por consola). Si hay 3 o menos coches, esperan a que haya sitio libre.
- MEJORA 6: No se puede acceder al parking hasta que el centro comercial esté abierto. El centro comercial se abre desde el hilo principal, cuando el usuario escribe por teclado cualquier mensaje con un Scanner
- MEJORA 7: Cada vez que entra o sale un coche del parking se deben mostrar las plazas libres que quedan.

5.a.3.En una tienda de mascotas están teniendo problemas para tener a todos sus hamsters felices.
Los hamsters viven una jaula en la que hay un comedero con comida y dos ruedas para hacer ejercicio.
Todos los hamsters quieren inicialmente comer del plato y, después, correr en la rueda.
Pero se encuentran con el inconveniente de que sólo cuatro de ellos pueden comer del comedero al mismo tiempo y sólo dos pueden correr en las ruedas.
El numero de hamsters se define en una constante (pongamos por ejemplo 20)
Cada hamster tarda entre 100 y 300 milisegundos en comer y unos 100 milisegundos en la rueda, tras lo cual, se pira a dormir.
Define un proceso que ejecuten los hamsters concurrentemente de forma que sincronicen estas actividades usando semáforos.

5.a.4.Hay 5 filósofos sentados entorno a una mesa circular. Estos filósofos se pasan la vida pensando y de vez en cuando comiendo. En el centro de la mesa hay un plato de arroz que nunca se acaba. El arroz se toma con palillos. Entre dos filósofos contiguos hay un solo palillo que es compartido por ambos. Para comer, cada filosofo necesita 2 palillos...

Comportamiento de un filósofo:

- Piensa
- Coge palillos
- Come
- Libera palillos
- Vuelve a pensar y empezar el ciclo



5.a.5.En una clínica de traumatológica hay tres secciones diferentes: TAC, Escayola y Rayos-X. Los enfermos acceden a la clínica para hacerse una de las tres pruebas, y esperan a que les atienda una enfermera que les indica la sala a la que deben acceder. Si la sala que necesitan esta llena, han de esperar en el pasillo a que haya sitio.

La sección TAC tiene una sala de espera para 5 enfermos, Escayola tiene una sala de espera para 6 enfermos y Rayos-X tiene sala de espera de 3 enfermos.
Posteriormente, cada enfermo de cada sala ha de esperar al único médico que hay de cada especialidad, y pasar por el médico en orden de entrada a la sala de espera, donde están 200 milisegundos, antes de volver a su casa.
Realizar un programa concurrente de forma que utilizando semáforos coordine la llegada de 40 enfermos a la clínica. Cada enfermo tiene que acudir a una especialidad (elegida al azar), y para que no lleguen todos a la vez, cada enfermo "viene paseando" un tiempo al azar antes de llegar a la clínica (entre 100 y 1000 milisegundos)

5.a.6.En un cuartel hay un comedor donde van a comer 100 soldados. Todos los soldados llegan a la vez al recinto par comer, y tratan de coger una bandeja de las únicas 10 que existen para tal efecto. Posteriormente se dirige a uno de los 4 mostradores donde le despachan la comida (en el mostrador solo puede haber un soldado a la vez), y si consigue un mostrador, cada soldado espera un tiempo x en recibir la comida. Luego pasa un tiempo al azar mientras come. Cuando finaliza, devuelve la bandeja sale del recinto. Realizar un programa concurrente de forma que utilizando semáforos coordine las tareas de los soldados.

- MEJORA 1: Cuando ya tienen comida y van a sentarse a comer, siempre intentan coger silla con respaldo, pero solo hay 3 sillas con respaldo, así que si no pueden coger silla con respaldo, cogen un taburete (de los que hay 300). Cuando un soldado acaba de comer, libera la silla con respaldo o el taburete que hubiera usado.
- MEJORA 2: Cuando se coge la bandeja, se debe coger bebida. Los soldados siempre intentan coger refresco, pero solo hay 50 refrescos. Si no pueden coger refresco, cogen un vaso de agua. Pero si pueden coger refresco, el soldado necesita uno de los 3 abridores existentes. Tarda 200 ms en usar el abridor, El abridor se devuelve cuando el soldado termina de comer.
- MEJORA 3: En cada mostrador puede haber hasta dos soldados.

5.a.7.Implementar un programa que ejecute correctamente la sincronización de tres procesos (P1, P2 y P3) de tal forma que se establezca el orden de ejecución P1, P2 y P3. Esto es, primero se ejecuta P1 y cuando finaliza P1 se puede ejecutar P2, y cuando finaliza P2 se puede ejecutar P3 y cuando finaliza P3 se puede ejecutar P1 y así sucesivamente.

5.a.8. En un pabellón de deportes existen 5 pistas para jugar a baloncesto. Evidentemente, para jugar un encuentro se precisa de un balón. Existe una caja donde están todos los balones. El delegado de campo coloca 3 balones en dicha caja. A la hora de apertura del pabellón van llegando los equipos para jugar. Cuando hay dos equipos disponibles para jugar cogen un balón de la caja (si hay). Sincronizar, utilizando semáforos, los partidos a celebrarse, imaginando la llegada de 20 equipos (cada equipo tarda un tiempo x en llegar (tiempo aleatorio), para que no lleguen todos a la vez). Cuando hay dos equipos que pueden jugar, inician un partido que dura x segundos (tiempo aleatorio), tras los cuales devuelven el balón a su cajón inicial. Los equipos que no tienen balón esperan a que otros partidos terminen.

5.a.9. Ejercicio Barbería versión 2. En una barbería hay un peluquero, que tiene una silla para afeitar a los clientes y 5 sillas para que esperen los clientes.

Para hacer que los clientes no lleguen todos a la vez, cada cliente esta "paseando" un tiempo al azar antes de ir a la barbería (entre 100 y 1000 milisegundos, al azar)
Si llegan clientes se sientan si hay sillas libres, si no, esperan en la calle a que haya sitio libre.
Cada cliente tarda en pelarse entre 300 y 1000 milisegundos, al azar.
Hacer que se creen tantos clientes como un valor que se pida al usuario por consola al inicio del programa.
Definir los semáforos necesarios y su inicialización.
En el programa se debe ir indicando por consola las acciones de los clientes: cuándo llegan, cuándo comienzan a pelarse, cuándo se van...

- MEJORA 1: Cuando llegan los clientes a la barbería, se sientan si hay sillas libres, o se van si no pueden sentarse
- MEJORA 2: Cada cliente se gasta en el peluquero entre 30 y 60 euros. Hace que se creen 40 clientes que intentan pelarse y decir al final cuanto ha ganado el peluquero

5.a.10. La cadena de montaje. En una cadena de montaje existe un robot encargado de colocar productos de 3 tipos diferentes ("alicate", "tenaza" o "punzón") en la cadena de montaje. Otros robots, retiran los productos de la cadena de montaje para realizar su empaquetado, teniendo en cuenta que están especializados en un solo tipo de producto ("alicate", "tenaza" o "punzón"), ignorando los que no son de su tipo. Finalmente, se quiere llevar un control del total de productos empaquetados (independientemente de su tipo).

Modelar usando semáforos o como un productor-consumidor el sistema descrito con las siguientes indicaciones:

- Modelar cada robot como un hilo: habrá en total 2 robots colocadores y 3 robots empaquetadores (un empaquetador para cada tipo de producto).
- Los productos son colocados de uno en uno en la cadena, y solamente en posiciones libres (se puede considerar que en la cadena de montaje caben un máximo N de elementos). Si no hay posiciones libres el robot colocador tendrá que esperar hasta que algún producto sea retirado de la cadena.
- Los robots empaquetadores se especializan su tipo de producto en tiempo de inicialización.
- Los robots empaquetadores comprueban si hay algún elemento de su tipo en la cadena ignorando los productos que no sean de su tipo. Si hay algún producto de su tipo lo retiran de la cadena (sólo 1 producto cada vez) y la posición queda libre para colocar nuevos productos, en caso contrario se quedan a la espera de que haya nuevos productos.
- Los robots empaquetadores de distinto tipo pueden funcionar a la vez.
- Tanto el colocador como los empaquetadores acaban cuando los conocedores han puesto 50 productos cada uno.
- Cada vez que un robot empaquetador procesa un producto, la cuenta total de productos empaquetados debe aumentar y mostrarse un mensaje por pantalla.

- Al final, se debe mostrar cuántos productos de cada tipo se han producido por los productores.

5.a.11. Cruce con semáforos. Sea un cruce de calles, por el que circulan coches de oeste a este y de norte a sur. Para regular el tráfico hay dos semáforos, uno en la entrada oeste y otro en la entrada norte.

Se desea desarrollar un monitor en Java que simule la gestión de los semáforos de la siguiente forma:

- Los coches se modelan como hilos que invocan un método llegaPorElNorte() o llegaPorElOeste() cuando llegan al cruce. Se crean 20 coches que aleatoriamente entran por el norte o por el oeste
- Si el semáforo correspondiente está en verde, el coche pasa inmediatamente. Si está en rojo, espera hasta que esté verde.
- Los coches tardan un cierto tiempo en pasar el cruce. Al salir invocan un método sale() en el monitor.
- Un hilo de control que gestiona los dos semáforos y llama a un método cambiaSemáforos() cada cierto tiempo para cambiar la configuración de los semáforos, dejando siempre uno abierto y el otro cerrado, alternando esta situación.

5.a.12.Los pacientes del hospital. Hacer un programa en el que 10 pacientes acuden a un hospital , siguiendo estas reglas:

- Cada paciente tarde un tiempo aleatorio entre 0 y 5 segundos en llegar al hospital
- El hospital solo tiene una sala de acceso, donde caben solo 4 pacientes. Si los pacientes que llegan al hospital no caben en la sala de acceso, esperan en la calle.
- Cuando un paciente entra en la sala de espera, elige (al azar) a que especialidad quiere acudir, siendo las especialidades posibles "Enfermeria", "Pediatria" y "Medicina General"
  - Para la especialidad de Enfermería, existen solo 2 consultas. Por ello, no puede atender más que a 2 pacientes a la vez.
  - Para la especialidad de Pediatría, existen solo 2 consultas. Por ello, no puede atender más que a 2 pacientes a la vez.
  - Para la especialidad de Enfermería, existen solo 1 consulta. Por ello, no puede atender más que a 1 pacientes a la vez.
- Los pacientes que entran en una consulta están un tiempo aleatorio entre 1 y 10 segundos en la misma, y luego abandonan el hospital

Realizar el ejercicio usando un **array de semáforos** con los que considere necesarios.

5.a.13.Carreras de remo. Hay 10 equipos de remo que comienzan una carrera, donde la finalizar la misma hay una meta marcada con un banderín.

Todos los equipos arrancan, y tardan un tiempo al azar entre 100 y 1000 milisegundos en llegar donde esta el banderín.

Usando semáforos, hacer que cada hilo informe de como llegan a la meta :

- el equipo que primero llegue al meta, debe escribir "Somos el equipo XX y hemos ganado !!!" (sustituir XX por el nombre del equipo)
- el resto de equipos, deben escribir "Somos el equipo XX, hemos llegado tarde.... en el puesto ZZZ" (sustituir ZZZ por el puesto en que hayan llegado)

5.a.14. La tienda generosa

Una tienda ofrece 10 cajas-regalo a los primeros 10 clientes que pasen a recogerlas

El encargado de la tienda saca del almacén una caja-regalo cada 200 milisegundos y la pone en el mostrador de la tienda.

En el mostrador no caben más de 2 cajas, así que si ya hay dos, el encargado debe esperar a que alguien se lleva alguna caja.

La noticia la conocen 20 clientes, que salen corriendo desde sus casas a la tienda

Tardan en llegar un tiempo al azar entre 400 y 600 milisegundos

Según llegan, y en el mismo orden en que van llegando, se ponen en cola y van a ir cogiendo los regalos según estén disponibles en el mostrador
Cuando un cliente coge un regalo, muestra en consola "Soy el cliente x y he conseguido regalo" y terminan su ejecución.
Los clientes que no consigan regalo, muestran "Soy el cliente x y no he conseguido regalo, me piro a casa", y terminan su ejecución.

5.a.15. La clínica veterinaria. En una clínica veterinaria se realizan operaciones a gatos y perros. Los perros y gatos al llegar a la clínica pasan por el preoperatorio antes de entrar al quirófano. El funcionamiento de la clínica es el siguiente:

- Los gatos y perros son hilos.
- Se espera la llegada de 2 gatos y de 5 perros a la clínica.
- Al preoperatorio sólo pueden entrar 5 animales.
- A la sala de operaciones pueden entrar 2 perros a la vez, pero cuando entra un gato no puede haber otro animal en el quirófano.
- El preoperatorio de los gatos dura 3 segundos y el de los perros dura 2 segundos (simular el tiempo utilizando un sleep).
- La operación en el quirófano dura para los gatos 2 segundos y para los perros dura 3 segundos (simular el tiempo utilizando un sleep).
- Para controlar el acceso y salida al preoperatorio y al quirófano se deben utilizar semáforos.

MEJORA 1: Nos han informado que el Gato2 y el Perro3 entran por urgencias por lo que debemos dentro de lo posible darles mayor prioridad frente al resto de pacientes.

MEJORA 2: Todos los animales llegan al hospital en el mismo instante. Para implementarlo vamos a hacer un arranque sincronizado utilizando CountDownLatch.

5.a.16. Ejercicio Barbería versión 3. El barbero dormilón (the sleeping barber, problema original de Edsger Dijkstra). Una barbería tiene una sala de espera con N sillas, más la silla de barbero. Si no hay clientes, el barbero se echa a dormir. Si un cliente entra en la barbería y encuentra todas las sillas ocupadas, se marcha de la barbería. Si el barbero está ocupado, pero hay sillas disponibles, el cliente se sienta en una de ellas. Si el barbero está dormido, el cliente lo despierta. Se trata de escribir un programa que coordine al barbero y los clientes, cada uno de los cuales se corresponde con un hilo.

5.a.17. Ejercicio Trámites Ayuntamiento de Pocosquedamos. Se necesita gestionar los de trámites del ayuntamiento de este pueblo.
Los 20 ciudadanos del municipio donde se encuentra el ayuntamiento entran por la puerta y piensan durante dos segundos en qué cola situarse de las tres que existen (todas las colas valen para todos los trámites).
Aleatoriamente elegirá una de ellas.
Cada una de las colas dirige a una ventanilla donde los ciudadanos pueden realizar uno de estos dos trámites: alta padrón o pago tributos.
Cuando el ciudadano llega a la ventanilla, el funcionario le dice las siguientes frases: “Buenos días señor/a X, soy el funcionario número Y (dónde X e Y son números) ¿Desea realizar el alta en el padrón o pago de tributos?”
El ciudadano elegirá aleatoriamente la operación que quiere realizar.
Si la operación es el pago de tributos, además, se le pide al ciudadano que indique el importe que va a pagar.
Cuando el ciudadano haya elegido la operación, el funcionario tarda un tiempo aleatorio (de 2 a 5 segundos) en hacer efectiva la operación.

A continuación, el ciudadano le dice al funcionario la siguiente frase: “Muchas gracias funcionario Y”.
Contar con que debe existir una variable que indique el número de altas de padrón que se ha realizado, el número de pagos de tributos realizados y el importe total de éstos.
Cuando los 20 habitantes han pasado por el ayuntamiento, se informa por consola de los tres valores indicados (número de altas de padrón, el número de pagos realizados y el importe total de éstos).

5.b. EJERCICIOS NIVEL MEDIO

5.b.1. Hacer un programa que simule un partido de futbol entre dos equipos.

La victoria dependerá de los goles que marquen ambos equipos, en el que gana el mejor de 9 goles, esto es, el primer equipo que marque 5 goles gana el juego.
Habrá un árbitro que dará la orden de comienzo de juego y reanudación tras marcar un gol.
Para la simulación, se utilizarán dos hilos que imprimirán por pantalla un identificador de hilo (El nombre del equipo), y el número de goles que lleva marcados el equipo
Para que ambos hilos tengan igualdad de oportunidades y para simular el juego, se generará aleatoriamente en cada turno un tiempo de espera diferente a cada hilo de no más de 2 segundos para que meta el gol el más rápido.
Para simular la señal del árbitro que inicia el partido, y cada turno para obtener un nuevo gol, el programa esperará la pulsación de cualquier tecla y Enter
Se tendrá que realizar la implementación correcta de semáforos y la escritura simultánea del resultado en consola (opcionalmente se escriben tambien los resultados en un fichero).

5.b.2. El puente estrecho.

Construya un sistema para gestionar el tráfico sobre un puente estrecho. El puente lo pueden atravesar vehículos desde ambos extremos. Como el puente tiene un solo carril, en un momento dado sólo puede haber vehículos cruzándolo en un único sentido. El puente tiene una capacidad de carga limitada: si hay más de tres vehículos, se hunde. Implemente una solución en la que cada vehículo sea un hilo. El algoritmo que sigue cada vehículo debe ser parecido a esto:

```
void vehículo (sentido) {
   llegar_al_puente(sentido)
   cruzar_puente(sentido)
   salir_del_puente(sentido)
}
```

El parámetro sentido indica en qué sentido se cruza el puente (o sea, que puede tener dos valores, por ejemplo izquierda y derecha).
Garantice que no ocurren colisiones y que el puente no se hunde.
Como añadido opcional, intente que no se produzcan esperas indefinidas.

5.b.3. Problema del baño mixto

Ante la carencia de baños en el Edificio de Consultoría, el administrador ha decidido que los pocos baños sanos sean de uso mixto: pueden entrar tanto hombres como mujeres, pero con la condición de que simultáneamente sólo pueda haber personas de un único sexo. Implemente la gestión de uno de estos baños (cada persona se simula como un hilo que entra y sale del baño).

- MEJORA 1: el baño tiene una capacidad limitada.
- MEJORA 1: diseñar una solución libre de inanición

5.b.4. Caníbales y misioneros

En el África profunda viven junto a un río una tribu de caníbales y un grupo de misioneros. El río se puede cruzar mediante un bote, con capacidad para tres personas. Como las aguas son bravas, el bote siempre tiene que transportar carga completa. Si se sientan dos misioneros y un caníbal, éstos le comen

el tarro al caníbal, hasta el punto de que el caníbal se arroja al agua: hay que evitar esta situación. Las demás combinaciones no dan problemas.
El sistema debe lanzar varios hilos, uno por cada persona. Escriba dos procedimientos: LlegaMisionero() y LlegaCaníbal(), para cuando cada persona llega al bote. Estos procedimientos dejan al hilo esperando hasta que el bote se encuentre lleno y seguro. Cuando esto ocurra, el procedimiento retorna y se considera que el barco vuelve a quedar vacío.

5.b.5. Radiado atómico (atomic broadcast)

Tenemos un proceso productor y 3 procesos consumidores que se comunican a través de un búfer de capacidad C. Cada elemento producido va destinado a todos los consumidores: un elemento en el búfer no desaparece hasta que ha sido leído por todos los consumidores. Cada consumidor va leyendo los elementos producidos de forma secuencial, a su propio ritmo: puede haber consumidores mucho más adelantados que otros.

6. EJERCICIOS DE INTERRUPCIONES

6.a.1. Realizar el ejercicio 2_a_5 (carreras de relevos) sin usar join(), utilizando interrupciones para que cada hilo avise a otro de cuando ha de empezar

7. EJERCICIOS DE SOCKETS

7.a. **SOCKETS TCP (con UN SOLO cliente)**

7.a.1.Mensajes simples.

Un cliente se conecta a un servidor. El cliente va pidiendo palabras al usuario por consola, y las envía luego al servidor. El servidor por cada palabra que recibe devuelve dos mensajes: el numero de letras que tiene la palabra y la palabra convertida en mayusculas.
El cliente recibe los dos mensajes del servidor, y los pinta en consola.
Todo termina cuando el cliente escribe fin o FIN y lo envía al servidor

7.a.2.Ejercicio concurso de frutas.

Inicialmente, servidor tiene almacenado en un array el nombre de 5 frutas.
Luego se conecta un cliente al servidor. El cliente escribe por consola una fruta y se la envía al servidor, quien responde al cliente si la fruta recibida se encuentra entre las 5 que tiene guardadas ("Acertaste!"), o si falló y no esta entre las 5 guardadas ("Fallaste !!")
Este proceso se repite 3 veces, y al final de 3 intentos, el servidor le envía a cliente el total de aciertos que ha obtenido.

- MEJORA 1: No permitir que el usuario repita la misma fruta mas de una vez, en tal caso, se considera la respuesta incorrecta

7.a.3.Ejercicio examen.

Se conectan dos ordenadores (llamémosles examinador y alumno). El examinador tiene dos arrays, uno con preguntas y otro array con las respuestas a las preguntas (obviamente, de igual tamaño). Un ejemplo puede ser esto:

```
String[] preguntas = {
"Elementos de los que esta compuesto un byte",
"Nombre del metodo que inicia un programa java",
"Interfaz de la que hereda ArrayList",
"Tipo primitivo para almacenar un entero lo mas grande posible",
"Clase para arrancar un servidor de sockets",
"Tipo de sockets que no pierden la conexion"
};

String[] respuestas = {
"bit",
"main",
"List",
"long,
"ServerSocket",
"TCP"
};
```

El proceso simple es : el examinador hace una pregunta (escogida al azar) al alumno, el alumno contesta con lo que cree que es la respuesta (se pide el usuario que indique por consola la contestación), y el examinador le indica si ha acertado o no
Este proceso se repite 3 veces (o las que se quiera), y al final , le examinador le dice al alumno cuantas preguntas ha acertado.
No considerar mayusculas o minúsculas en las respuestas del alumno

- MEJORA 1: No permitir que el servidor haga mas de una vez la misma pregunta

7.a.4.Adivinanza.

Se conectan dos ordenadores (llamémosles servidor y concursante).
El servidor "imagina" un numero al azar entre 0 y 100

El concursante envía un número al servidor como intento de adivinar el que el servidor ha "imaginado"
El servidor le responde con un "acertaste" si es en verdad el número buscado, o envía "Mayor" si su número imaginado es mayor que el que ha propuesto el concursante, o envía "Menor" si su número imaginado es menor que el que ha propuesto el concursante
Este proceso se repite hasta que el concursante acierta el número, en cuyo momento el servidor envía al concursante el número de intentos que ha hecho para acertar, y el concursante escribe dicho número en consola

MEJORA 1: Los mensajes van encriptados con cifrado asimétrico. Se supone que tanto servidor como cliente compartes las mismas claves, que podemos tener en una clase común con métodos y atributos estáticos.

- NOTA: cuidado al enviar y recibir arrays de bytes por un socket: ver Anexos del capitulo de "Criptografía" para aprender como hacerlo correctamente

7.a.5.Factura (versión 1)

Hay un servidor TCP y un cliente asociado. El cliente va a pedir al servidor que le calcule el importe total de una factura. Para ello, el cliente va a enviar al servidor 5 mensajes, y cada mensaje tiene a su vez dos datos: el primer dato es el nombre de un producto, y el segundo, el importe del mismo. El servidor debe ir calculando la suma de los importes de los productos. Para ello, si el nombre del producto empieza por "A", el importe del producto lo incrementa con un IVA del 10%, y si no, lo incrementa con un IVA del 21%. Al final, tras recibir los 5 mensajes, el servidor envía de vuelta al cliente el importe total de la factura

7.a.6.Submarinos Version 2

Estamos en la segunda guerra mundial, y los submarinos alemanes envían mensajes a su centro de control indicando sus próximos movimientos.

Los submarinos envían mensajes ficticios y validos aleatoriamente. Todos los mensajes (ficticios y reales) se extraen de este array de mensajes:

```
String [] mensajes = {"vamos al norte", "vamos al sur","yendo al sur","yendo al norte","mas al sur","giro al sur","no nos movemos", "sin avanzar"};
```

El centro de control hace de servidor, al que se conecta 1 submarino.
El submarino envía 10 mensajes al centro de control, un mensaje cada 200 milisegundos, mensajes que extraen aleatoriamente del array de mensajes.
Cuando el controlador recibe un mensaje, si es ficticio lo añade a una lista, y si es verdadero, lo añade a otra lista distinta.Los controladores saben que un mensaje es cierto porque contiene la letra "m", son falsos los que no la contienen.
Al final del proceso, el controlador muestra por consola las dos listas, la de mensajes correctos y la de los mensajes incorrectos que ha recibido de su submarino correspondiente.

7.a.7.Censura.

Imaginemos un servidor que hace censura en los envíos de mensajes. Al servidor se le conectan dos ordenadores clientes (llamémosles ClienteA y ClienteB). El ClienteA quiere enviar mensajes al ClienteB, pero los debe hacer pasando por el servidor. Así, para enviar una mensaje, ClienteA lo envía al servidor, y este lo envía a ClienteB. Entre medias, el servidor va a modificar el mensaje, de modo que si en el mismo encuentra palabras de menos de 4 letras, las va a modificar por "****". Así, si el ClienteA envía como mensaje "Buenos días, ¿que tal estas?" el ClienteB debe recibir "Buenos días, ¿**** **** estas?". El proceso termina cuando el ClienteA escriba un mensaje que diga "FIN", mensaje que SI se envía al otro cliente, que en este caso debe ver "FIN" (no debe ver ***). Todos terminan cuando se procesa el mensaje FIN.
Los mensajes que el clienteA envia se piden por Scanner
Tener en cuenta que los dos clientes hacen cosas distintas, y que el servidor necesita conectar !! con ambos clientes a la vez !!

7.b. **SOCKETS TCP (multicliente)**

7.b.1.Contador de conexiones

Escribe un programa que cuente el número de conexiones que vaya recibiendo. Este programa dispondrá de un socket servidor. Cada vez que un socket cliente se conecte, el servidor le enviará un mensaje con el número de clientes conectados hasta ahora. Así pues, el primer cliente que se conecte recibirá un 1, el segundo un 2, y así sucesivamente. Luego, además, el servidor pide por consola 5 números, que ira enviando al cliente. El cliente al final le contesta con la suma de los números recibidos.

7.b.2. Comeletras.

El servidor construye inicialmente un arraylist con 10 de las letras del abecedario en mayusculas
Cada cliente tiene un proceso que repetirá 5 veces;
- pide por consola por scanner una letra.
- La envia al servidor
- El servidor recoge la letra enviada, y hace estas pruebas:
  * Si la letra no esta en su arraylist, avisa al cliente con el mensaje "Fallaste".
  * Si la letra está en su arraylist de letras, la elimina del arraylist y avisa al cliente con el mensaje "Acertaste".
  * Si ademas de acertar, la lista queda vacía, el mensaje del servidor al cliente es "FINALIZASTE", lo que hace que el cliente termine su proceso.

A partir de ese momento, si algún cliente aun sigue enviando letras al servidor, este siempre contesta a los clientes con el mensaje "YA SALIO UN GANADOR", lo que hace que los otros clientes tambien terminen.
TENER SIEMPRE EN CUENTA QUE EL ARRAY DEL SERVIDOR ES EL MISMO PARA TODOS LOS CLIENTES

7.b.3.Votaciones Festival.

Simularemos los votos del festival de música de HeuristicVision.
Tendremos de partida una clase PaisParcipante que tiene como atributos el nombre del país y los votos que puede obtener.
El servidor tiene un array con los 10 PaisParticipante que compiten en el festival, ya creado e inicializado.
El servidor va recibiendo llamadas de varias naciones votantes (un cliente TCP), que le ofrecen sus votaciones.
Cada cliente (nación votante) envía al servidor el nombre de su propia nación, el nombre del país participante al que vota, y los votos que le otorga, y ahí termina su ejecución como cliente. La información que envía al servidor se le pide al usuario por consola
El servidor recibe la información , y la contabiliza en el país correspondiente
Tras recibir los votos de diez naciones, el servidor muestra el resultado final, indicando quien ha ganado el festival
Nota: Un país no puede votarse a si mismo, ni a un país inexistente, si el servidor lo detecta, anula los votos enviados

7.b.4.Duelo de clientes (versión 1).

Un servidor escoge un número al azar entre 0 y 100 , y espera que ese conecten 4 clientes.
Por otro lado, 4 clientes se conectan a este servidor, escribiendo al servidor primero su nombre, y luego un número que ellos creen que es el inventado por el servidor
El servidor crea 4 hilos, uno para cada cliente, procesa la recepción de cada nombre y cada sugerencia de número, y luego contesta cual de los 4 clientes ha acertado el número que inicialmente pensó, y si ninguno acertó, que cliente ha quedado mas cerca

- MEJORA 1: Para tratar a los clientes, el servidor los procesa en hilos independientes del hilo servidor.

7.b.5. Duelo de clientes (versión 2).

Un servidor elige un numero al azar entre 0 y 100 (llamémosle número "escondido") y espera a que dos clientes conecten con él. Cada cliente, tras conectarse, envía 5 números al servidor (los elige al azar). El servidor lee los 5 número de cada cliente (da igual el orden), y calcula cuál de los números esta más cerca del número "escondido", decidiendo cual de los dos clientes es el "ganador". Informa posteriormente a cada cliente si ha ganado o ha perdido. En caso de empate, se consideran a ambos clientes "ganadores". El servidor va mostrando por consola su número "escondido" y los números que recibe, así como la decisión de qué cliente es el ganador.

- MEJORA 1: Para tratar a los clientes, el servidor los procesa en hilos independientes del hilo servidor.

7.b.6.Envío de ficheros.

El objetivo del ejercicio es crear una aplicación cliente/servidor que permita el envío de ficheros de texto al cliente. Para ello, el cliente se conectará al servidor por el puerto 2223 y le solicitará al usuario por la consola el nombre de un fichero del servidor.
Luego, el cliente le manda al servidor el nombre del fichero.
Si el fichero existe, el servidor, le enviará el fichero al cliente y éste lo mostrará por pantalla y creará un fichero en local con el mismo nombre que el que está en el servidor.
Para realizarlo ten en cuenta:
El servidor abrirá el fichero y enviará línea a línea el contenido del fichero al cliente.
Cuando termine de enviar todas las líneas, el servidor enviará al cliente un mensaje de que ya ha terminado de enviar las líneas (EOF).
El cliente, leerá las líneas enviadas por el servidor y las mostrará y guardará en un fichero por pantalla hasta que se reciba el mensaje EOF.

Si el fichero no existe, el servidor le enviará al cliente un mensaje de error.
Una vez que el cliente ha mostrado el fichero se finalizará la conexión.

7.b.7. Submarinos Version 3

Se trata de ampliar el ejercicio “Submarinos versión 2”, incluyendo la opción de multicliente.
Estamos en la segunda guerra mundial, y los submarinos alemanes envían mensajes a su centro de control indicando sus próximos movimientos.

Los submarinos envían mensajes ficticios y validos aleatoriamente. Todos los mensajes (ficticios y reales) se extraen de este array de mensajes:

```
String [] mensajes = {"vamos al norte", "vamos al sur","yendo al sur","yendo al norte","mas al sur","giro al sur","no nos movemos", "sin avanzar"};
```

El centro de control hace de servidor, al que se conectan 3 submarinos.
Los 3 submarinos envían 10 mensajes al centro de control, un mensaje cada 200 milisegundos, mensajes que extraen aleatoriamente del array de mensajes.
El centro de control crea varios hilos (controladores) para atender a los diferentes submarinos
Cuando el controlador recibe un mensaje, si es ficticio lo añade a una lista, y si es verdadero, lo añade a otra lista distinta (ambas listas son comunes a todos los controladores). Los controladores saben que un mensaje es cierto porque contiene la letra "m", son falsos los que no la contienen.
Al final del proceso, cada controlador muestra por consola las dos listas, la de mensajes correctos y la de los mensajes incorrectos que se han recibido de todos los submarinos.

- MEJORA 1: Cuando un submarino envía tres mensajes inválidos seguidos, el controlador envía al submarino el mensaje "ABORTAR", y termina la comunicación entre controlador y submarino.

- MEJORA 2: Los submarinos envían los mensajes codificados con un algoritmo AES (simétrico), que los controladores deben desencriptar
  - NOTA: cuidado al enviar y recibir arrays de bytes por un socket: ver Anexos del capitulo de "Criptografía" para aprender como hacerlo correctamente

7.b.8. El Banco Central de Morosolandia

El Banco Central de Morosolandia tiene un deposito donde guarda los lingotes de Mithril que posee el estado. Ademas, este banco tiene un servidor que recibe las conexiones de los camiones que meten y sacan lingotes en el Banco
Los camiones que sacan lingotes
- piden al usuario por consola la cantidad de lingotes que quieren sacar
- conectan con el servidor,
- dicen su nombre,
- dicen que son "de los que sacan Mithril",
- indican la cantidad de lingotes quieren sacar.

El banco envía un mensaje al camión indicándole que le permite sacarlos y luego los saca
(eso se simula rebajando el total de lingotes que tiene el banco, nada más. El camión simplemente recibe el mensaje, informa por su consola, y acaba.)
Cuidado que SOLO permite sacar al camión lo que pida, si tiene lingotes suficientes

Los camiones que meten lingotes
- piden al usuario por consola la cantidad de lingotes que quieren sacar
- conectan con el servidor,
- dicen su nombre,
- dicen que son "de los que meten Mithril",
- indican la cantidad de lingotes quieren meter.

El banco envía un mensaje al camión indicándole que le permite meterlos y luego los mete
(eso se simula aumentando el total de lingotes que tiene el banco, nada más. El camión simplemente recibe el mensaje, informa por su consola, y acaba.)

Tras cada operación de un camión, meter o sacar, el banco informa de la operación y del estado final en que queda el deposito de Mithril.

NOTA INFORMATIVA:

En definitiva, todo el ejercicio se trata de un productor consumidor, puede haber varios hilos sacando y metiendo elementos a la vez, con lo que debería tenerse en cuenta que los accesos a la zona crítica son controlados con métodos sincronizados y con wait y notifyall …

MEJORA 1

- los camiones que sacan deben identificarse y mandar una palabra clave justo después de indicar su nombre. El servidor comprueba que esa palabra clave esta dentro de un array de claves que él tiene ya predefinido desde el inicio.

MEJORA 2

- Si algún camión de los que quieren sacar lingotes no acierta con la clave, el servido cancela TODOS los futuros camiones mientras investiga, y la investigación dura un tiempo de 15 segundos. Si alguien intenta conectar durante la investigación, su petición es rechazada y no se le atiende

MEJORA 3

- El banco además de lingotes de Mithril, guarda lingotes de Kryptonita y de Adamantium. Hay por lo tanto camiones que ingresan y camiones que sacan cualquiera de las 3 cosas, y el estado tiene depósitos de las 3 cosas. Controlar además que cada uno de los tres depósitos tiene una capacidad máxima.

7.b.9. La liga de Balonmano. Se desea crear un pequeño servidor de datos que acepte todas las conexiones posibles en el cual se pueda almacenar información sobre equipos de balonmano. La información que se debe almacenar es un código autoincremental, nombre del equipo, puntuación y categoría.
Un cliente puede enviar al servidor varias cosas:
* Puede enviar una línea con el texto «AGREGAR». En ese caso el cliente debe enviar después tres líneas:
  - La primera línea contiene el nombre del equipo
  - La segunda línea la puntuación.
  - La tercera línea la categoría.
* Un cliente puede enviar la cadena «CONSULTAR».
En ese caso el servidor contesta con una número que indica cuántos equipos tiene almacenados, y posteriormente, envía al cliente todas la informaciones de cada equipo, cada dato en un mensaje.
Si contesta por ejemplo que hay 2 equipos, mas tarde debe enviar 2 cuartetos de líneas (código, nombre, puntuación y categoría).
Si contesta por ejemplo que hay 4 equipos, mas tarde debe enviar 4 cuartetos de líneas (código, nombre, puntuación y categoría).

7.c. SOCKETS UDP

7.c.1.Con Sockets UDP, escribe un programa que responda a saludos usando sockets datagram. El programa escuchará por el socket mensajes que contengan la cadena de texto “Hola”. Cuando reciba uno, responderá a su emisor con otro mensaje que contenga la cadena “¿Qué tal?”. Escribe además un programa adicional para probar el funcionamiento de este.

7.c.2.Alumnos (versión 1). Con sockets UDP, se crea inicialmente una clase Alumno con atributos nombre (String) y edad (int). Se crea luego un proceso emisor que crea un objeto de la clase Alumno y se lo envía a otro proceso receptor, quien envía de vuelta un texto al emisor indicándole si el alumno esta jubilado o no (imaginamos la edad de jubilación a los 70)

7.c.3.Alumnos (versión 2). Con sockets UDP, y usando la clase Alumno del ejercicio anterior (con atributos nombre (String) y edad (int). Se crea una clase Escuela, que tiene como atributos un ArrayList de alumnos. Se crea luego un proceso emisor que crea un objeto de la clase Escuela, y añade varios alumnos a la escuela, y se envía la escuela a otro proceso receptor, quien evalúa el array de alumnos de la escuela, elimina los alumnos que son menores de edad, y devuelve el objeto de Escuela al emisor con el array revisado. El Emisor escribe por consola el contenido del array recibido de alumnos de la escuela

7.c.4. Mensaje Circular
Crear una programa en el que varios clientes (programas) se vayan enviando un mensaje (token), en forma de anillo. Cada uno de los clientes se debe arrancar indicando como argumentos de entrada su posición en el anillo, y el tamaño del anillo. El cliente que se encuentre en la posición número 1 (el primero), generará el mensaje “token” y se lo enviará al cliente 2. Cuando este lo reciba se lo pasará al 3, y así sucesivamente. Cuando lo reciba el último cliente, lo enviará de vuelta al número 1. Cuando el número 1 lo reciba, la secuencia se interrumpirá (el token habrá dado una vuelta completa al anillo).
Se deben cumplir además las siguientes restricciones:
- Todos los clientes deben tener el mismo código fuente. Se trata, por tanto, del mismo programa, pero ejecutado con distintos parámetros.
- El programa debe permitir un número variable de elementos en el anillo.
- El tamaño del anillo se especificará de antemano.

- Se puede programar usando sockets stream (en este caso los mensajes pasan por el servidor) o sockets UDP.

7.d. EJERCICIOS NIVEL MEDIO

7.d.1. El buzón de mensajes:

Con sockets TCP (con servidor-multiclientes), crear un par de programas que simulen un Buzón de mensajes para varios clientessiguiendo el modelo cliente/servidor. Los clientes se conectan al servidor, indicando su nombre. Posteriormente, el cliente podrá consultar si tienen mensajes para él, o dejar mensajes para todos los otros usuarios (los que se hayan conectado alguna vez). El servidor almacenará los mensajes entregados hasta que el usuario al que van dirigidos los consulte, momento en que desaparecen del buzón. Consejo: hacer una clase Mensaje, con atributos destinatario, remitente y mensaje. Gestionar los mensajes con un arraylist o hashset de objetos Mensaje. Gestionar además todos los clientes que se van conectando en un arraylist de strings con los nombres de los clientes

7.d.2. El Casino en casa

Se van a simular de modo simple una única apuesta de un casino en una ruleta "modesta". Para ello el programa se construirá con la siguientes características:

- Existirá un casino, que tiene un croupier (el servidor), al que se conectan cuantos clientes se desee.
- El casino comienza con un saldo de 2000 y cada cliente con un saldo propio de 100
- La apuesta debe hacerse en un intervalo de tiempo concreto, pasado el cual no se aceptan más apuestas (más clientes) (como cuando el croupier dice eso de "no va más").
- Luego los diversos hilos apuestan para ver si aciertan, enviando su apuesta al crupier, que será un número del 1 al 10 (el número que apuestan) y la cantidad que apuestan (deben tener ese dinero en su saldo.)
- Luego, el croupier 'saca' un número al azar entre el 0 y el 10. Si algún cliente ha acertado el número, le informa al cliente que su saldo ha aumentado lo que ha apostado * 10. Si sale el 0, todo el mundo pierde y la banca se queda con todo el dinero apostado.
- Posteriormente, el cliente corta la conexión con el servidor, y se le permite que nuevamente se vuelva a conectar para la siguiente apuesta. Los hilos del servidor, por lo tanto, acaban tambien.
- El croupier vuelve a iniciar un periodo de espera de clientes, y vuelve a empezar todo el ciclo.
- La banca acepta todas las apuestas pero nunca paga más dinero del que tiene, si se queda sin dinero la partida termina, por lo que el crupier nunca caba de jugar, salvo que el casino se quede sin dinero
- Cuando los jugadores pierden todo su dinero, terminan de jugar
- Los jugadores no pueden hacer apuestas si no tienen dinero suficiente. Si un jugador no tiene dinero suficiente se retira.

7.d.3.Copia Carpeta.

Con sockets UDP, crea un proceso A que lea los ficheros de un directorio y los envía una a uno a otro programa B. El primer envío de A no es un fichero, sino la cantidad de ficheros que va a enviar posteriormente, para que B sepa cuántos ficheros ha de leer. B ha de guardar todos los ficheros en su propio directorio. Todos los ficheros han de tener menos de 1024 bytes.

7.d.4.Mini Test,

Con sockets TCP o UDP, pero siempre leyendo byte a byte.

Escribe un programa que conteste a preguntas. El programa creará un socket stream y aguardará conexiones. Cuando llegue una conexión, leerá los mensajes recibidos, byte a byte, hasta que encuentre el carácter ASCII "?" (signo de final de interrogación).

Cuando esto ocurra, construirá una frase con todos los bytes recibidos y contestará con un mensaje. El contenido del mensaje será siempre de tipo String y dependerá de la frase recibida:

- Si la frase es "¿Como te llamas?", responderá con la cadena "Me llamo ejercicio 2".

- Si la frase es “¿Cuántas líneas de código tienes?, responderá con el mensaje “Tengo xxx líneas” (xxx es el número de líneas de código que tenga)
- Si la frase es cualquier otra cosa, responderá “No he entendido la pregunta”.

Escribe, además, dos programas de prueba que verifiquen que el programa responde bien a todas las preguntas.

8. EJERCICIOS DE CRIPTOGRAFÍA

8.a.1. Realizar un programa que pida varias líneas de texto por consola, las guarde en un arraylist, y luego escriba el arraylist en un fichero de texto. Crear posteriormente un nuevo fichero encriptando el fichero anterior, con un algoritmo simétrico. Posteriormente, desencriptar el fichero encriptado y comprobar línea a línea que son las mismas que hay en el arraylist inicial

8.b. EJERCICIOS NIVEL MEDIO

8.b.1. La clase concurso tiene un arraylist de palabras. Con sockets TCP, un concursante se conecta a la clase concurso. El concursante trata de adivinar 5 veces alguna de la palabras del array, para lo cual, para cada palabra:

- el "concursante" pide la usuario por consola la palabra
- la envia al concurso
- el concurso contesta al concursante "OK" o "MAL" , según si la palabra esta o no en su array
- Al final, el concursante muestra por pantalla cuantas palabras ha acertado

Eso sí, todos los envíos se han de encriptar con claves simétricas, en ambas direcciones.

- NOTA: cuidado al enviar y recibir arrays de bytes por un socket: ver Anexos del capitulo de "Criptografía" para aprender como hacerlo correctamente

8.b.2. El servidor del banco ChuperChanchullos tiene varios clientes, y para cada uno, tiene en su disco un fichero de movimientos bancarios del cliente, siendo el nombre del fichero el mismo que el del cliente acabado en .txt. Por ejemplo, para el cliente *PAGOPOCO*, existe un fichero llamado *PAGOPOCO.TXT* con un contenido parecido a este;

22-04-21 Mordida del alcalde 120000
22-04-21 Clavo del agente XXX 8000
22-04-21 Tapado de documentos 90600

El servidor del banco ChuperChanchullos recibe conexiones de varios clientes, via sockets TCP.
A cada cliente se le trata como un hilo independiente del servidor.
El cliente al conectarse le envía su nombre, encriptado con un algoritmo simétrico.
El servidor desencripta el nombre del cliente, busca en su disco el fichero de movimientos bancarios del cliente, lee su contenido linea a linea y, cada linea, la encripta y envía al cliente (usa el mismo algoritmo simétrico). Para saber que ha terminado de enviar todas las líneas del fichero, el servidor envía un último mensaje "ENVIO FINALIZADO"
El cliente lee todas las lineas que le envía el servidor, las muestra por consola, y al finalizar, le escribe al servidor un mensaje de despedida.
Cuando el servidor lee el mensaje de despedida, se corta la conexión (solo de ese cliente).

- NOTA: cuidado al enviar y recibir arrays de bytes por un socket: ver Anexos del capitulo de "Criptografía" para aprender como hacerlo correctamente

9. EJERCICIOS DE TEMATICA MULTIPLE

9.a.1. Realizar un programa que Simule una noche en un casino y que evalué los distintos beneficios de diversas estrategias de juego de ruleta.

La ruleta francesa es un juego en el que hay una ruleta con 37 números (del 0 al 36). Cada 3000 milisegundos el croupier saca un número al azar y los diversos hilos apuestan para ver si ganan. Todos los hilos empiezan con 1.000 euros y la banca (que controla la ruleta) con 50.000. Cuando los jugadores pierden dinero, la banca incrementa su saldo.

Se puede jugar a un número concreto. Habrá 4 hilos que eligen números al azar del 1 al 36 (no el 0) y restarán 10 euros de su saldo para apostar a ese ese número. Si sale su número su saldo se incrementa en 360 euros (36 veces lo apostado).

Se puede jugar a par/impar. Habrá 4 hilos que eligen al azar si apuestan a que saldrá un número par o un número impar. Siempre restan 10 euros para apostar y si ganan incrementan su saldo en 20 euros.

Se puede jugar a la «martingala». Habrá 4 hilos que eligen números al azar. Elegirán un número y empezarán restando 10 euros de su saldo para apostar a ese número. Si ganan incrementan su saldo en 360 euros. Si pierden jugarán el doble de su apuesta anterior (es decir, 20, luego 40, luego 80, y así sucesivamente)

La banca acepta todas las apuestas pero nunca paga más dinero del que tiene.
Si sale el 0, todo el mundo pierde y la banca se queda con todo el dinero.

9.a.2.Un banco necesita controlar el acceso a cuentas bancarias y para ello desea hacer un programa de prueba en Java que permita lanzar procesos que ingresen y retiren dinero a la vez y comprobar así si el resultado final es el esperado.

Se parte de una cuenta con 100 euros y se pueden tener procesos que ingresen 100 euros, 50 o 20. También se pueden tener procesos que retiran 100, 50 o 20 euros euros. Se desean tener los siguientes procesos:

- 40 procesos que ingresan 100
- 20 procesos que ingresan 50
- 60 que ingresen 20.

De la misma manera se desean lo siguientes procesos que retiran cantidades.

- 40 procesos que retiran 100
- 20 procesos que retiran 50
- 60 que retiran 20.

Se desea comprobar que tras la ejecución la cuenta tiene exactamente 100 euros, que era la cantidad de la que se disponía al principio. Realizar el programa Java que demuestra dicho hecho.

9.a.3.Sea un puente con capacidad para un vehículo y dos accesos: norte y sur. En caso de que haya vehículos intentando entrar por los dos accesos, debe entrar un vehículo por el extremo en el que haya más esperando (si el número de vehículos esperando en cada extremo es el mismo, no es necesario imponer un orden). En el caso de que intente entrar una ambulancia, tendrá prioridad sobre el resto de vehículos. No es necesario considerar el caso en que dos ambulancias intenten acceder simultáneamente al puente. Se pide desarrollar una clase monitor GestorPuente que gestione el acceso al puente, según la especificación previa. Este puede se un esqueleto de dicho monitor:

```
public class GestorPuente {
      . . . void entrarNorte () { . . .}
      . . . void entrarSur () { . . .}
      . . . void entrarAmbulancia () { . . .}
      . . . void salirPuente(){. . .}
```

```
}
```

9.a.4.Se quiere desarrollar un sistema para controlar la temperatura y el número de personas que se encuentran en una sala de un museo.

En condiciones normales, se permiten 50 personas en la sala. Si la temperatura sube por encima de un umbral (tUmbral = 30), se limita el número de personas a 35. Si cuando se detecta este suceso el número de personas en la sala es mayor que 35, no es necesario desalojarlas.

Cada persona se representa mediante un hilo. Además, hay un hilo único que mide periódicamente la temperatura de la sala y notifica su valor al sistema. Se pide desarrollar un monitor (GestorSala) que sincronice a los hilos que representan personas y al hilo que mide la temperatura, de acuerdo con las especificaciones anteriores.

No es necesario garantizar que el orden de acceso a la sala coincide con el orden de llegada a la puerta de entrada.

El monitor debe proporcionar los siguientes métodos:

```
public class GestorSala {
      ... void entrarSala()
      // se invoca cuando una persona // quiere entrar en la sala.
      ... void entrarSalaJubilado()
      // se invoca cuando una persona jubilada // quiere entrar en la sala.
      ... void salirSala()
      // se invoca cuando una persona, jubilada o no, // quiere salir de la sala.
      ... void notificarTemperatura(int temperatura)
      // lo invoca la hebra que mide la temperatura
      // de la sala para indicar el último valor medido.
```

MEJORA 1: Si una persona jubilada intenta entrar, tendrá prioridad frente al resto de personas que estén esperando

9.a.5. Desarrolle un monitor en Java que gestione el despegue de aviones y avionetas en un aeropuerto como se especifica a continuación:

Los aviones al despegar generan turbulencias, por lo que entre dos despegues consecutivos tiene que transcurrir un intervalo de tiempo mínimo:

- 600 milisegundos después del despegue de un avión.
- 400 milisegundos después del despegue de una avioneta.

Además, se debe impedir que despeguen consecutivamente dos avionetas si hay aviones esperando. No hay restricciones de este tipo respecto a los aviones.
El monitor responde al siguiente esquema:

```
class GestorDespegue { ...
  ...  despegarAvion() {...}
// lo invoca un avión cuando quiere despegar

  ...  despegarAvioneta() {...}
// lo invoca una avioneta cuando quiere despegar

  ...  autorizarDespegue() {...}
// lo invoca el temporizador (ver má adelante) // para indicar que ha transcurrido el intervalo //
mínimo desde el despegue anterior
}
```